

Query Performance Rewrites: What Works, What Doesn't, and Why

Adam Machanic

Data Person





improving performance and some best practices for SQL

The difference between a junior and a senior SQL developer

Query performance. Here are 5 steps that will make your SQL queries faster.

1. Indexing: **Indexing** conditions, F

2. **AVOID SELECT *

3. Efficient Joins

4. **Using Proper Data Types

5. Limit Data Retrieval

6. **Avoiding Functions in WHERE

7. **Avoid SELECT *

8. **Using Proper Data Types

9. **Avoiding Functions in WHERE

10. **Avoiding SELECT *

11. **Avoiding SELECT *

12. **Avoiding SELECT *

13. **Avoiding SELECT *

14. **Avoiding SELECT *

15. **Avoiding SELECT *

16. **Avoiding SELECT *

17. **Avoiding SELECT *

18. **Avoiding SELECT *

19. **Avoiding SELECT *

20. **Avoiding SELECT *

21. **Avoiding SELECT *

22. **Avoiding SELECT *

23. **Avoiding SELECT *

24. **Avoiding SELECT *

25. **Avoiding SELECT *

26. **Avoiding SELECT *

27. **Avoiding SELECT *

28. **Avoiding SELECT *

29. **Avoiding SELECT *

30. **Avoiding SELECT *

31. **Avoiding SELECT *

32. **Avoiding SELECT *

33. **Avoiding SELECT *

34. **Avoiding SELECT *

35. **Avoiding SELECT *

36. **Avoiding SELECT *

37. **Avoiding SELECT *

38. **Avoiding SELECT *

39. **Avoiding SELECT *

40. **Avoiding SELECT *

41. **Avoiding SELECT *

42. **Avoiding SELECT *

43. **Avoiding SELECT *

44. **Avoiding SELECT *

45. **Avoiding SELECT *

46. **Avoiding SELECT *

47. **Avoiding SELECT *

48. **Avoiding SELECT *

49. **Avoiding SELECT *

50. **Avoiding SELECT *

51. **Avoiding SELECT *

52. **Avoiding SELECT *

53. **Avoiding SELECT *

54. **Avoiding SELECT *

55. **Avoiding SELECT *

56. **Avoiding SELECT *

57. **Avoiding SELECT *

58. **Avoiding SELECT *

59. **Avoiding SELECT *

60. **Avoiding SELECT *

61. **Avoiding SELECT *

62. **Avoiding SELECT *

63. **Avoiding SELECT *

64. **Avoiding SELECT *

65. **Avoiding SELECT *

66. **Avoiding SELECT *

67. **Avoiding SELECT *

68. **Avoiding SELECT *

69. **Avoiding SELECT *

70. **Avoiding SELECT *

71. **Avoiding SELECT *

72. **Avoiding SELECT *

73. **Avoiding SELECT *

74. **Avoiding SELECT *

75. **Avoiding SELECT *

76. **Avoiding SELECT *

77. **Avoiding SELECT *

78. **Avoiding SELECT *

79. **Avoiding SELECT *

80. **Avoiding SELECT *

81. **Avoiding SELECT *

82. **Avoiding SELECT *

83. **Avoiding SELECT *

84. **Avoiding SELECT *

85. **Avoiding SELECT *

86. **Avoiding SELECT *

87. **Avoiding SELECT *

88. **Avoiding SELECT *

89. **Avoiding SELECT *

90. **Avoiding SELECT *

91. **Avoiding SELECT *

92. **Avoiding SELECT *

93. **Avoiding SELECT *

94. **Avoiding SELECT *

95. **Avoiding SELECT *

96. **Avoiding SELECT *

97. **Avoiding SELECT *

98. **Avoiding SELECT *

99. **Avoiding SELECT *

100. **Avoiding SELECT *

About Me?

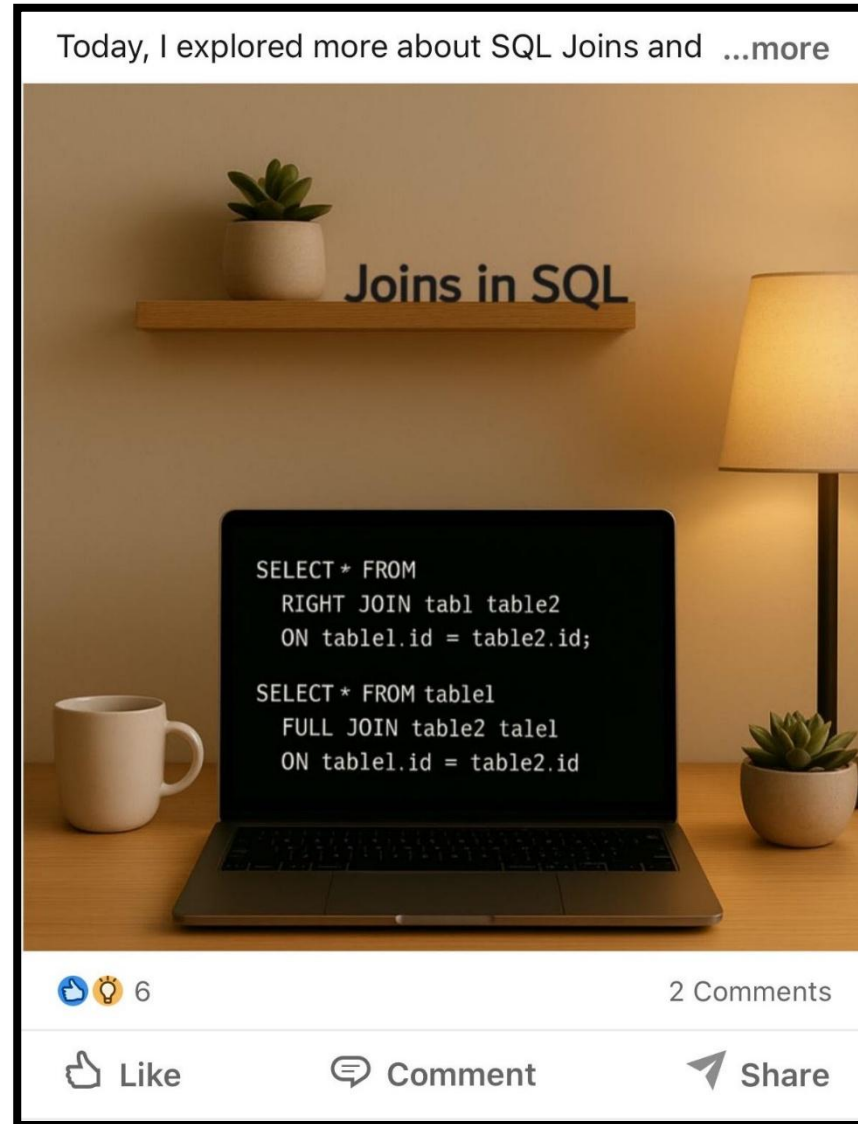


[linkedin.com/in/adammachanic](https://www.linkedin.com/in/adammachanic)



The Advice of fluencers

The Advice of fluencers



The Advice of influencers

“Use Indexes”

The Advice of influencers

“Use

“Don’t Use
SELECT *”

The Advice of influencers

“Use

“Do
SE

“Add TOP or
LIMIT to Reduce
Output Size”

The Advice of influencers

“Use

“Do
SE

“Add
LIMIT
Ou

“Don’t Use an
Unnecessary
Join”

The Advice of influencers

“Use

“Do
SE

“Add
LIMIT
Ou

“Don’t
Un

“Avoid Complex
Logic”

The Advice of influencers

“Use

“Do
SE

“Add
LIMIT
Ou

“Don’t
Un

“Avoid Complex
Logic”

Here’s a trick you should use
in all of your queries to make
them 10x faster, guaranteed!

There is No Quick Fix

There is No Quick Fix



We Want
Faster Queries!

There is No Quick Fix

We Want
Faster Queries!

We Don't Want
to Waste Time!

There is No Quick Fix

We Want
Faster Queries!

Solution:
Understand Why

We Don't Want
to Waste Time!

There is No Quick Fix

We Want
Faster Queries!

Solution:
Understand Why

We Don't Want
to Waste Time!

Take Actual
Control

There is No Quick Fix

We Want
Faster Queries!

Solution:
Understand Why

We Don't Want
to Waste Time!

Take Actual
Control

Eliminate
Random
Guessing

Query Rewrites: The Three Options

Query Rewrites: The Three Options

Modify the query with
no logical change,
within the optimizer's
model

Total waste of time

Query Rewrites: The Three Options

Modify the query with
no logical change,
within the optimizer's
model

Total waste of time

Inadvertently modify
the query's logic,
maybe making it
faster

You just created a bug

Query Rewrites: The Three Options

Modify the query with
no logical change,
within the optimizer's
model

Total waste of time

Inadvertently modify
the query's logic,
maybe making it
faster

You just created a bug

Speed up the logic by
making changes that
the optimizer can't

Winner!

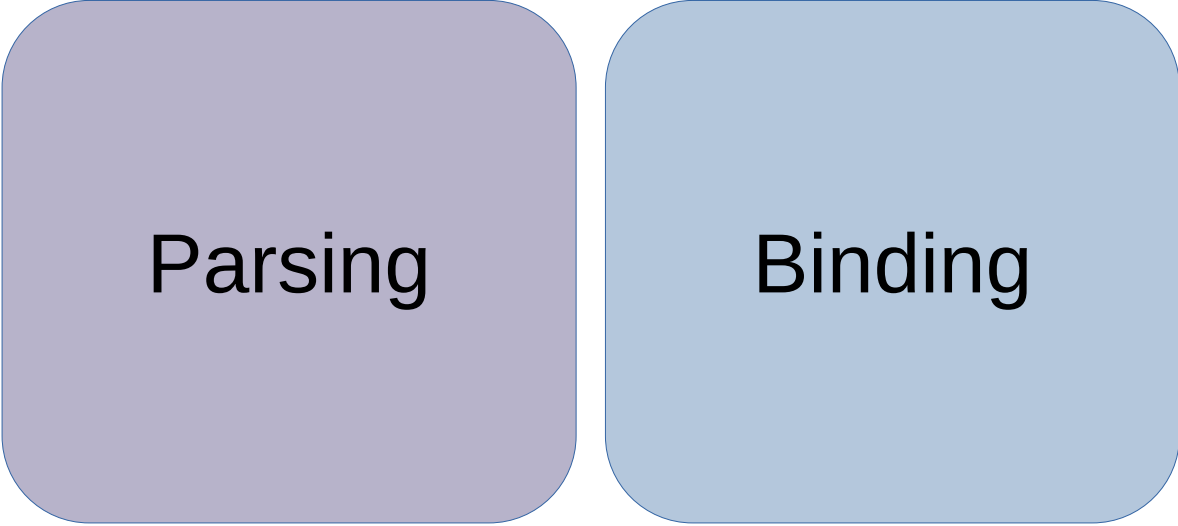
Before Your Query Runs

Before Your Query Runs



Parsing

Before Your Query Runs

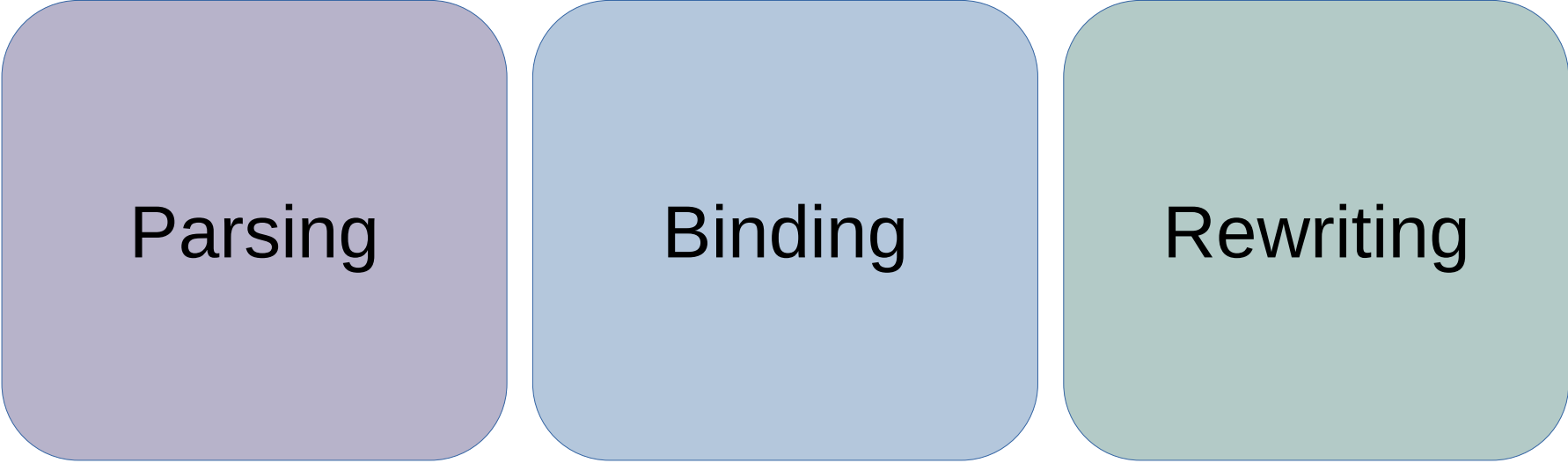


Parsing

The diagram consists of two rounded rectangular boxes placed side-by-side. The left box is a muted purple color and contains the word 'Parsing'. The right box is a light blue color and contains the word 'Binding'. Both boxes have a thin dark border and rounded corners.

Binding

Before Your Query Runs

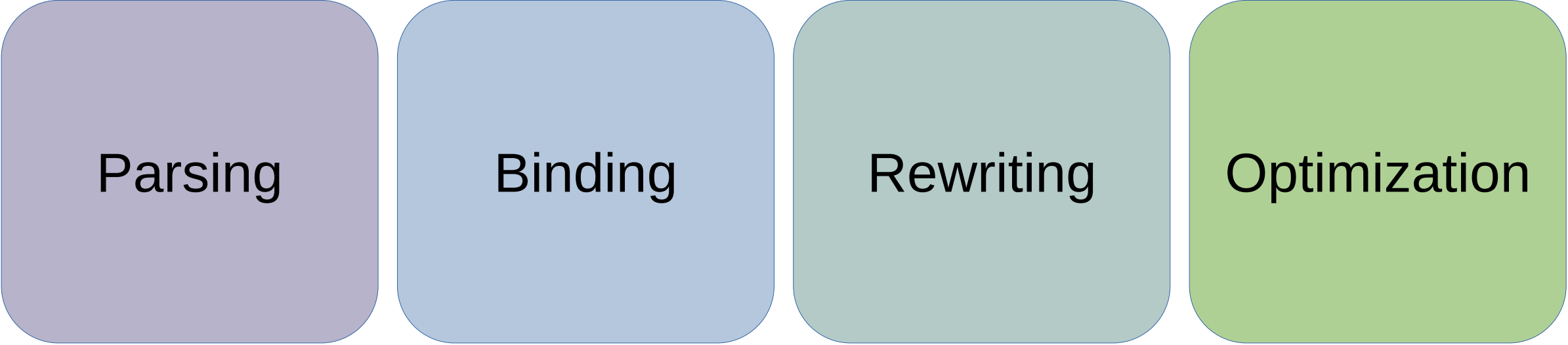


Parsing

Binding

Rewriting

Before Your Query Runs



Parsing

Binding

Rewriting

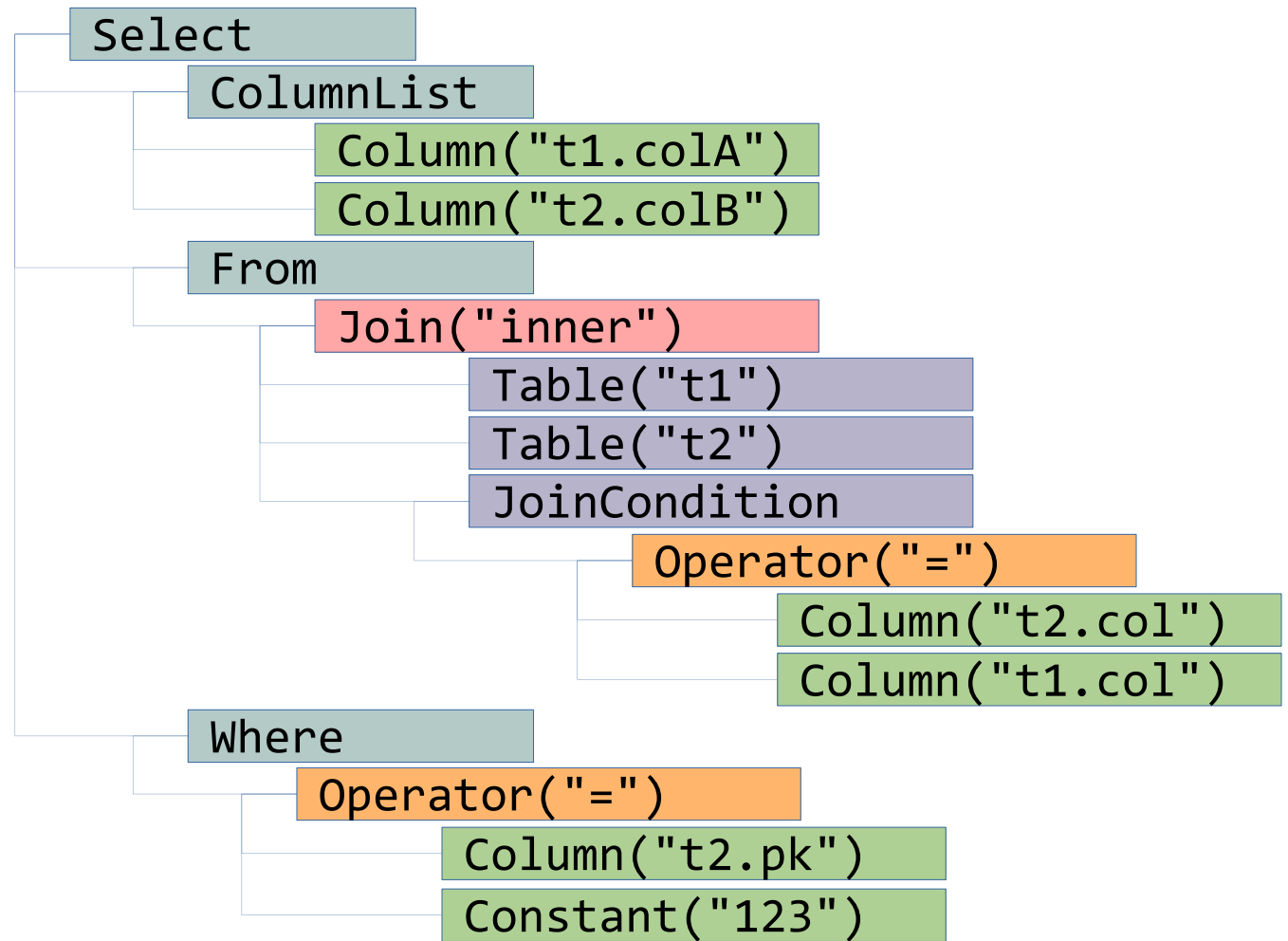
Optimization

SQL to Parse Tree

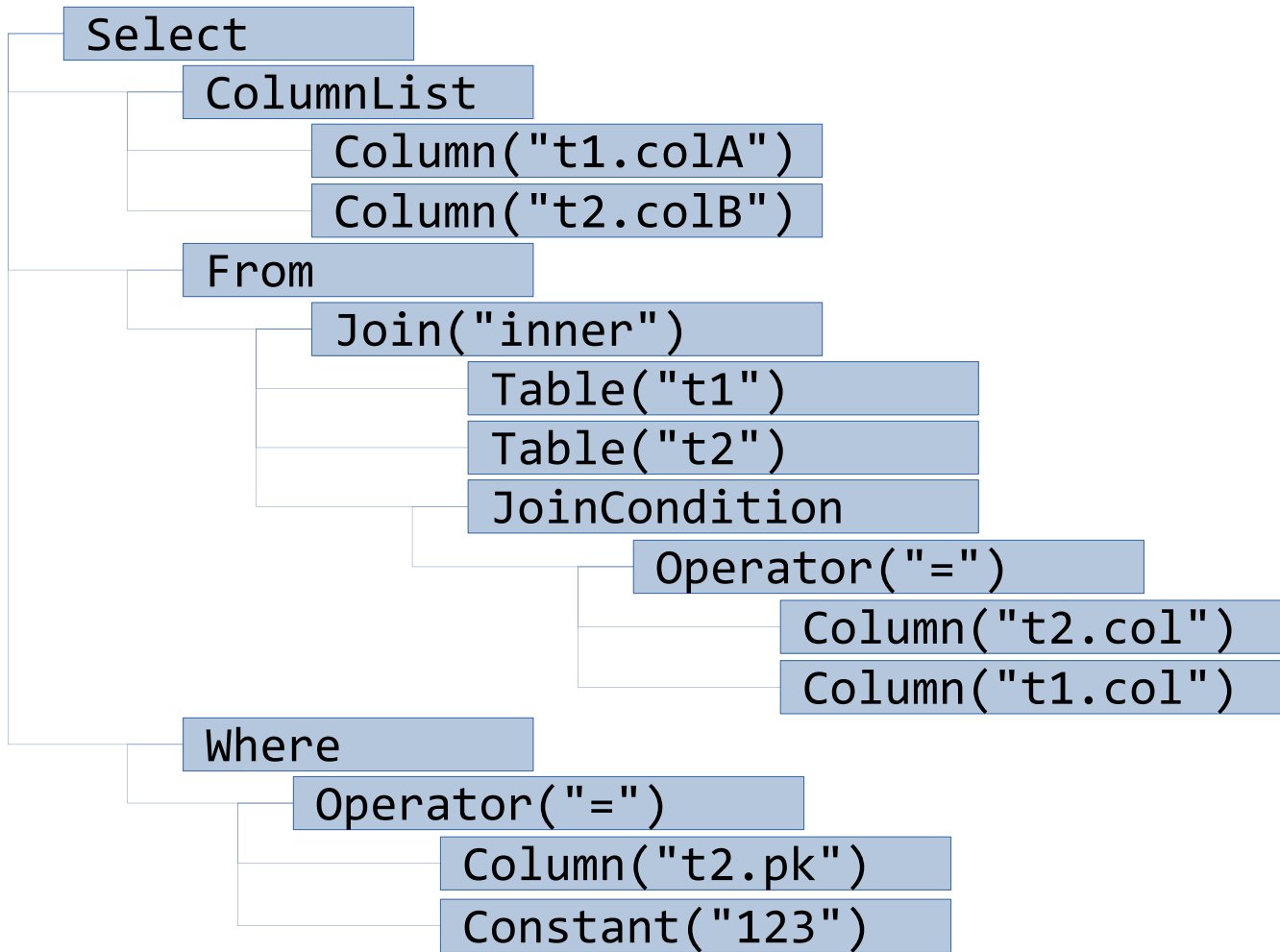
```
SELECT
    t1.colA,
    t2.colB
FROM t1
INNER JOIN t2 ON
    t2.col = t1.col
WHERE
    t2.pk = 123
```

SQL to Parse Tree

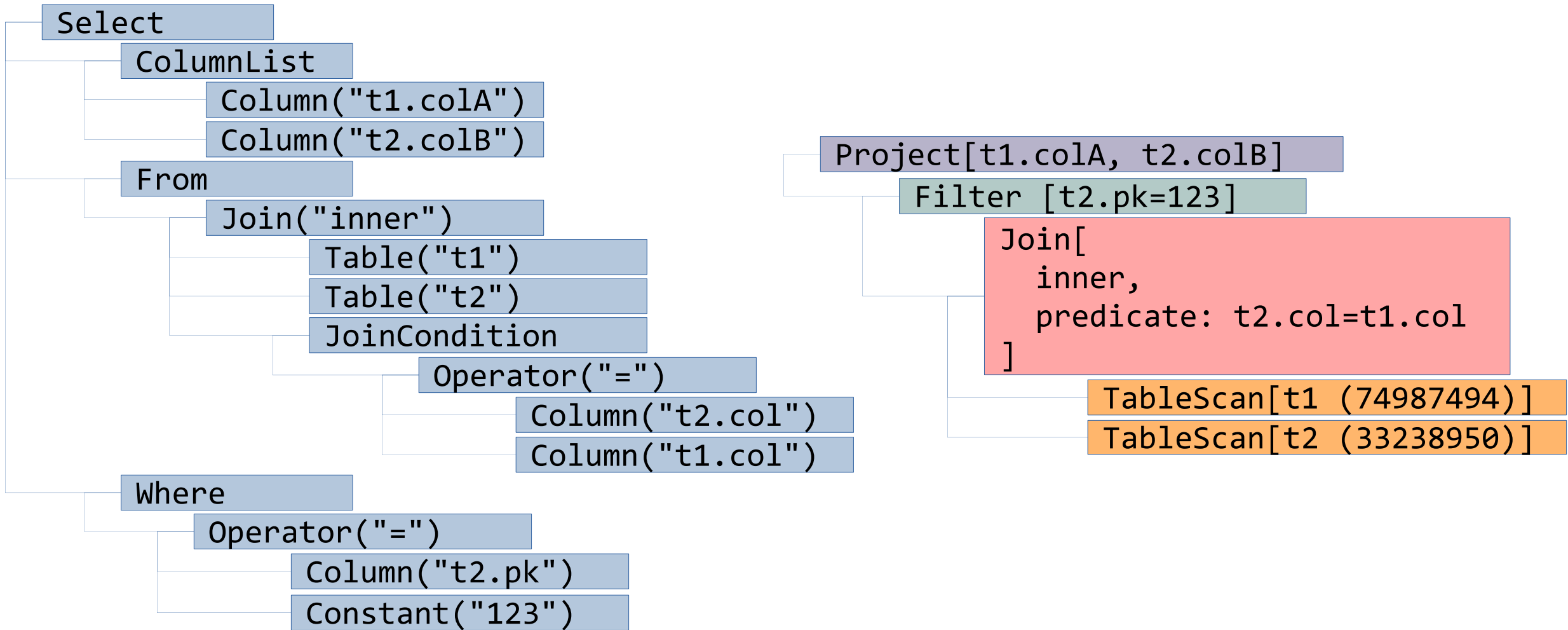
```
SELECT
    t1.colA,
    t2.colB
FROM t1
INNER JOIN t2 ON
    t2.col = t1.col
WHERE
    t2.pk = 123
```



Parse Tree to Logical Operator Tree



Parse Tree to Logical Operator Tree



Rewrite (Normalization)

Rewrite (Normalization)



Rules-based
changes, to prepare
for later cost-based
strategies

Rewrite (Normalization)

Rules-based
changes, to prepare
for later cost-based
strategies

Flattening
(View Expansion)

Decorrelation

Constant Folding

Simplification

Rewrite (Normalization)

Rules-based
changes, to prepare
for later cost-based
strategies

Flattening
(View Expansion)

Decorrelation

Constant Folding

Simplification

IMPORTANT

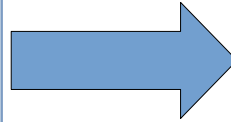
Operates on the
logical operator tree,
not on SQL!

View/Subquery/CTE Flattening

```
SELECT
    x.*
FROM tbl1 AS x
INNER JOIN
(
    SELECT
        y.*
    FROM tbl2 AS y
    WHERE
        y.pk = 123
) AS y0 ON
    y0.col = x.col
```

View/Subquery/CTE Flattening

```
SELECT
  x.*
FROM tbl1 AS x
INNER JOIN
(
  SELECT
    y.*
    FROM tbl2 AS y
    WHERE
      y.pk = 123
) AS y0 ON
  y0.col = x.col
```



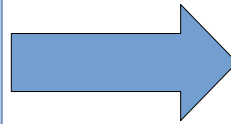
```
SELECT
  x.*
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  y.col = x.col
WHERE
  y.pk = 123
```

Subquery Decorrelation (semijoin)

```
SELECT
  x.*
FROM tbl1 AS x
WHERE
  EXISTS
  (
    SELECT
      y.*
    FROM tbl2 AS y
    WHERE
      y.col = x.col
  )
```

Subquery Decorrelation (semijoin)

```
SELECT
  x.*
FROM tbl1 AS x
WHERE
  EXISTS
  (
    SELECT
      y.*
    FROM tbl2 AS y
    WHERE
      y.col = x.col
  )
```



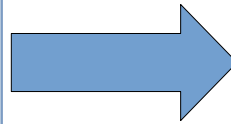
```
SELECT
  x.*
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  y.col = x.col
```


Subquery Decorrelation (antijoin)

```
SELECT
  x.*
FROM tbl1 AS x
WHERE
  NOT EXISTS
  (
    SELECT
      y.*
    FROM tbl2 AS y
    WHERE
      y.col = x.col
  )
```

Subquery Decorrelation (antijoin)

```
SELECT
  x.*
FROM tbl1 AS x
WHERE
  NOT EXISTS
  (
    SELECT
      y.*
    FROM tbl2 AS y
    WHERE
      y.col = x.col
  )
```



```
SELECT
  x.*
FROM tbl1 AS x
LEFT OUTER JOIN tbl2 AS y ON
  y.col = x.col
WHERE
  y.col IS NULL
```

Optimization and Exploration

Optimization and Exploration



Cost-based options
and decisions to try to
create the fastest plan

Optimization and Exploration

Cost-based options
and decisions to try to
create the fastest plan

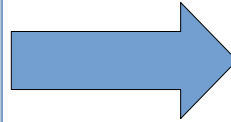
Reordering
Pushdown
Access Methods
Join Types

Join Reordering

```
SELECT  
    x.*  
FROM tbl1 AS x  
INNER JOIN tbl2 AS y ON  
    y.col = x.col
```

Join Reordering

```
SELECT
  x.*
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  y.col = x.col
```



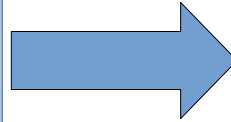
```
SELECT
  x.*
FROM tbl2 AS y
INNER JOIN tbl1 AS x ON
  x.col = y.col
```

Projection Pushdown

```
SELECT
    x.*, y.col2
FROM tbl1 AS x
INNER JOIN
(
    SELECT
        *
    FROM tbl2 AS y0
    WHERE
        y0.pk = 123
) AS y ON
    y.col = x.col
```


Projection Pushdown

```
SELECT
    x.*, y.col2
FROM tbl1 AS x
INNER JOIN
(
    SELECT
        *
    FROM tbl2 AS y0
    WHERE
        y0.pk = 123
) AS y ON
    y.col = x.col
```



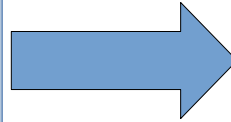
```
SELECT
    x.*, y.col2
FROM tbl1 AS x
INNER JOIN
(
    SELECT
        y0.col, y0.col2
    FROM tbl2 AS y0
    WHERE
        y0.pk = 123
) AS y ON
    y.col = x.col
```

Predicate Pushdown (inner join)

```
SELECT
  x.*
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  y.col = x.col
WHERE
  y.pk = 123
```

Predicate Pushdown (inner join)

```
SELECT
  x.*
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  y.col = x.col
WHERE
  y.pk = 123
```



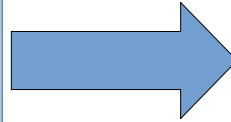
```
SELECT
  x.*
FROM tbl1 AS x
INNER JOIN
(
  SELECT
    *
  FROM tbl2 AS y0
  WHERE
    y0.pk = 123
) AS y ON
  y.col = x.col
```

Predicate Pushdown (outer-ish join)

```
SELECT
  x.*
FROM tbl1 AS x
LEFT OUTER JOIN tbl2 AS y ON
  y.col = x.col
WHERE
  y.pk = 123
```

Predicate Pushdown (outer-ish join)

```
SELECT
  x.*
FROM tbl1 AS x
LEFT OUTER JOIN tbl2 AS y ON
  y.col = x.col
WHERE
  y.pk = 123
```



```
SELECT
  x.*
FROM tbl1 AS x
INNER JOIN
(
  SELECT
    *
  FROM tbl2 AS y0
  WHERE
    y0.pk = 123
) AS y ON
  y.col = x.col
```

Predicate Pushdown (real outer join)

```
SELECT
  x.*
FROM tbl1 AS x
LEFT OUTER JOIN tbl2 AS y ON
  y.col = x.col
WHERE
  y.pk IS NULL
  OR y.pk = 123
```

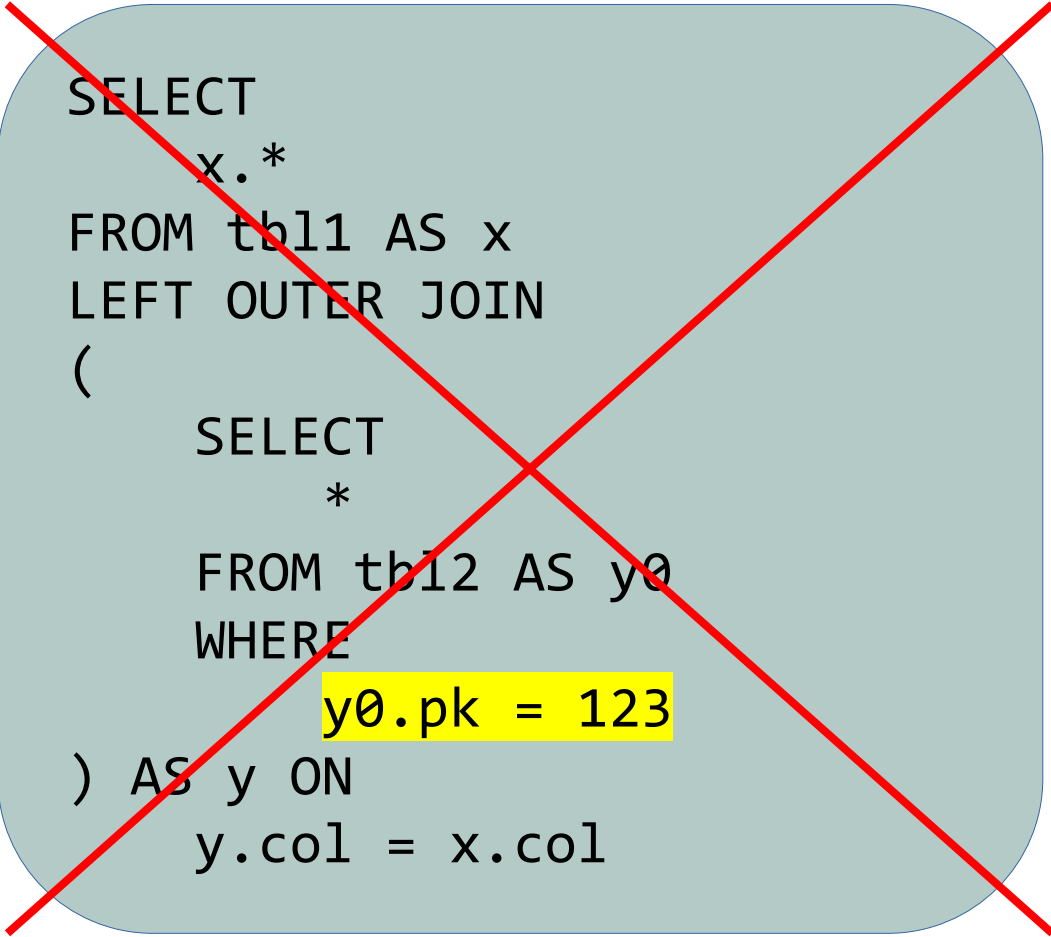
Predicate Pushdown (real outer join)

```
SELECT
    x.*
FROM tbl1 AS x
LEFT OUTER JOIN tbl2 AS y ON
    y.col = x.col
WHERE
    y.pk IS NULL
    OR y.pk = 123
```

```
SELECT
    x.*
FROM tbl1 AS x
LEFT OUTER JOIN
(
    SELECT
        *
    FROM tbl2 AS y0
    WHERE
        y0.pk = 123
) AS y ON
    y.col = x.col
```

Predicate Pushdown (real outer join)

```
SELECT
    x.*
FROM tbl1 AS x
LEFT OUTER JOIN tbl2 AS y ON
    y.col = x.col
WHERE
    y.pk IS NULL
    OR y.pk = 123
```



```
SELECT
    x.*
FROM tbl1 AS x
LEFT OUTER JOIN
(
    SELECT
        *
    FROM tbl2 AS y0
    WHERE
        y0.pk = 123
) AS y ON
    y.col = x.col
```


Why Can't We Push Down?

```
SELECT
  x.*
FROM Transaction AS x
LEFT OUTER JOIN Product AS y ON
  y.ProdId = x.ProdId
WHERE
  y.Name IS NULL
OR y.Name = 'foo'
```

Why Can't We Push Down?

```
SELECT
  x.*
FROM Transaction AS x
LEFT OUTER JOIN Product AS y ON
  y.ProdId = x.ProdId
WHERE
  y.Name IS NULL
OR y.Name = 'foo'
```

Transaction

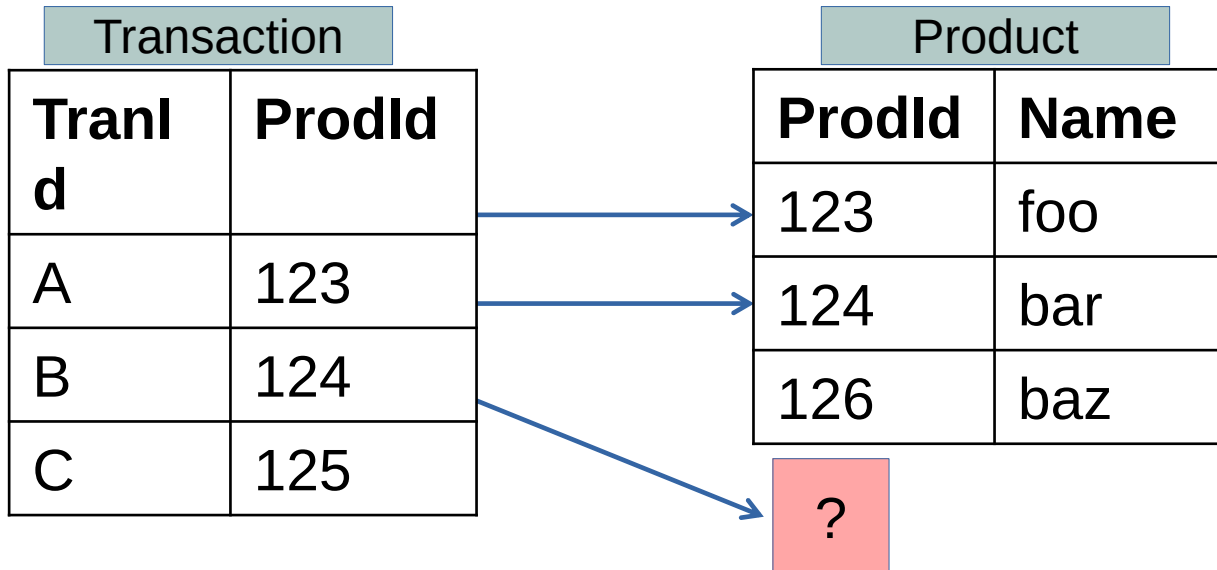
TransId	ProdId
A	123
B	124
C	125

Product

ProdId	Name
123	foo
124	bar
126	baz

Why Can't We Push Down?

```
SELECT
  x.*
FROM Transaction AS x
LEFT OUTER JOIN Product AS y ON
  y.ProdId = x.ProdId
WHERE
  y.Name IS NULL
  OR y.Name = 'foo'
```



Why Can't We Push Down?

```
SELECT
  x.*
FROM Transaction AS x
LEFT OUTER JOIN Product AS y ON
  y.ProdId = x.ProdId
WHERE
  y.Name IS NULL
OR y.Name = 'foo'
```

Before WHERE

TranId	ProdId	Name
A	123	foo
B	124	bar
C	125	[NULL]

Transaction

TranId	ProdId
A	123
B	124
C	125

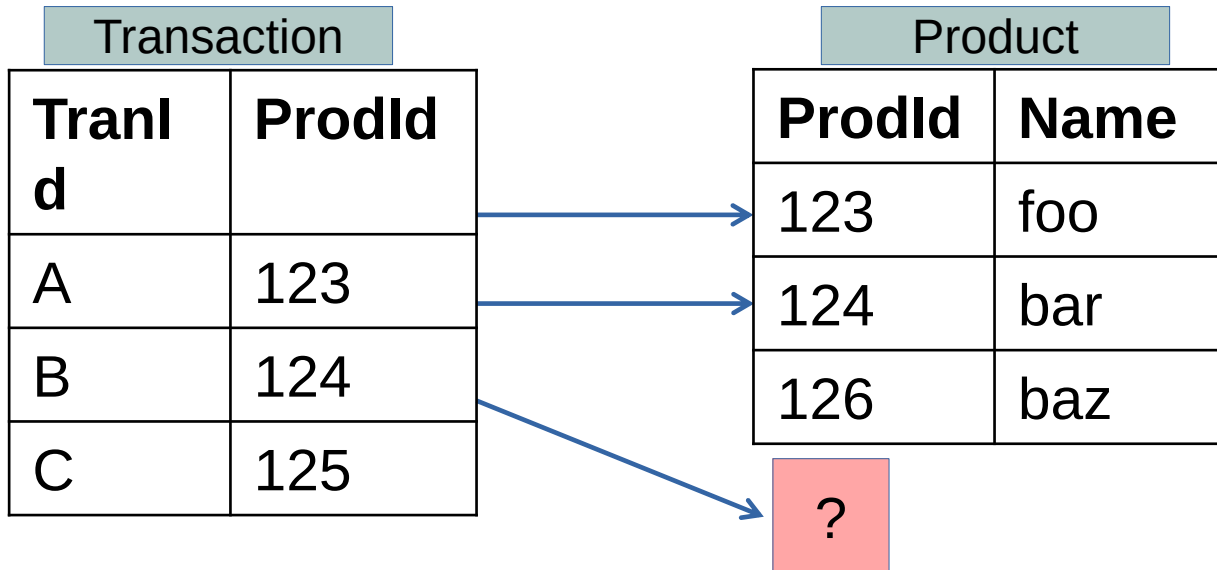
Product

ProdId	Name
123	foo
124	bar
126	baz

?

Why Can't We Push Down?

```
SELECT
  x.*
FROM Transaction AS x
LEFT OUTER JOIN Product AS y ON
  y.ProdId = x.ProdId
WHERE
  y.Name IS NULL
OR y.Name = 'foo'
```



Before WHERE

TranId	ProdId	Name
A	123	foo
B	124	bar
C	125	[NULL]

After WHERE

TranId	ProdId	Name
A	123	foo
C	125	[NULL]

Why Can't We Push Down?

```
SELECT
  x.*
FROM Transaction AS x
LEFT OUTER JOIN
(
  SELECT
    *
  FROM Product AS y0
  WHERE
    y0.Name = 'foo'
) AS y ON
  y.ProdId = x.ProdId
```

Why Can't We Push Down?

```
SELECT
  x.*
FROM Transaction AS x
LEFT OUTER JOIN
(
  SELECT
    *
  FROM Product AS y0
  WHERE
    y0.Name = 'foo'
) AS y ON
  y.ProdId = x.ProdId
```

Transaction

TransId	ProdId
A	123
B	124
C	125

Product

ProdId	Name
123	foo
124	bar
126	baz

Why Can't We Push Down?

```
SELECT
  x.*
FROM Transaction AS x
LEFT OUTER JOIN
(
  SELECT
    *
  FROM Product AS y0
  WHERE
    y0.Name = 'foo'
) AS y ON
  y.ProdId = x.ProdId
```

Transaction

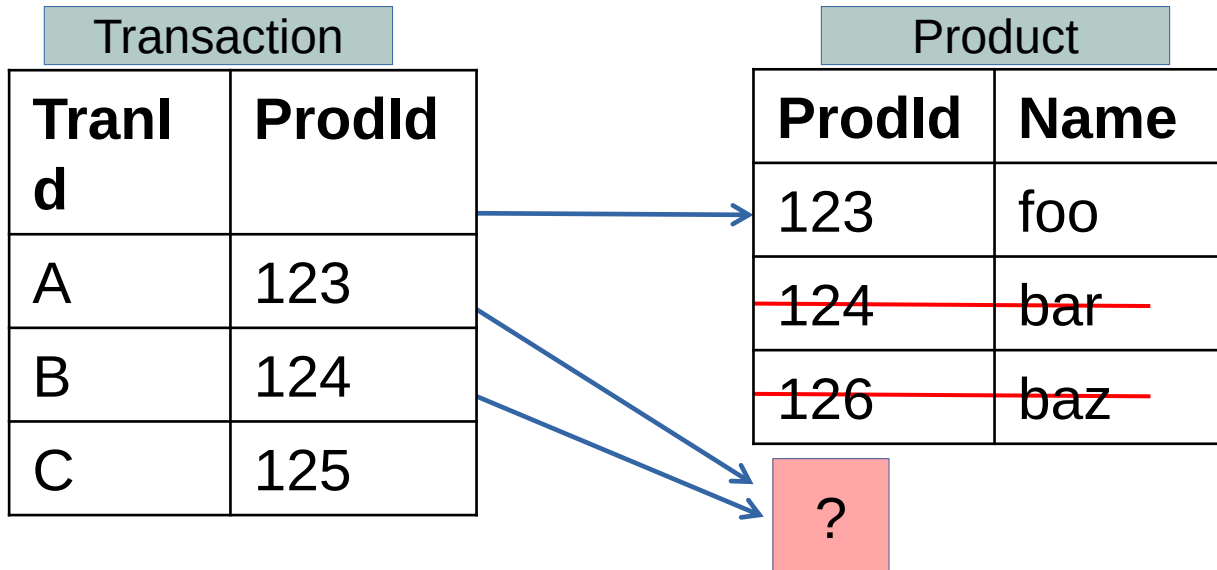
TransId	ProdId
A	123
B	124
C	125

Product

ProdId	Name
123	foo
124	bar
126	baz

Why Can't We Push Down?

```
SELECT
  x.*
FROM Transaction AS x
LEFT OUTER JOIN
(
  SELECT
    *
  FROM Product AS y0
  WHERE
    y0.Name = 'foo'
) AS y ON
  y.ProdId = x.ProdId
```



Why Can't We Push Down?

```
SELECT
  x.*
FROM Transaction AS x
LEFT OUTER JOIN
(
  SELECT
    *
  FROM Product AS y0
  WHERE
    y0.Name = 'foo'
) AS y ON
  y.ProdId = x.ProdId
```

Transaction

TranId	ProdId
A	123
B	124
C	125

Product

ProdId	Name
123	foo
124	bar
126	baz

?

Output

TranId	ProdId	Name
A	123	foo
B	124	[NULL]
C	125	[NULL]

in

wisdom



Examples Tested and Rendered With...

**AdventureWorks
SQL Server 2022 (SSMS)
Postgres 17 (explain.dalibo.com)**

in wisdom : Semijoins

in | **wisdom** : Semijoins

**Use EXISTS
instead of IN**

“It’s much faster.”

in | **wisdom** : Semijoins

**Use EXISTS
instead of IN**

“It’s much faster.”

**Don’t use
EXISTS or IN**

“INNER JOIN
scales better”

Use EXISTS rather than IN

```
SELECT
  x.*
FROM tbl1 AS x
WHERE
  x.col IN
  (
    SELECT
      y.col
    FROM tbl2 AS y
    WHERE
      y.pk = 123
  )
```


Use EXISTS rather than IN

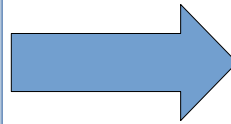
```
SELECT
  x.*
FROM tbl1 AS x
WHERE
  x.col IN
  (
    SELECT
      y.col
    FROM tbl2 AS y
    WHERE
      y.pk = 123
  )
```



**DOES IT
FLATTEN?**

Use EXISTS rather than IN

```
SELECT
  x.*
FROM tbl1 AS x
WHERE
  x.col IN
  (
    SELECT
      y.col
    FROM tbl2 AS y
    WHERE
      y.pk = 123
  )
```



```
SELECT
  x.*
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  y.col = x.col
WHERE
  y.pk = 123
```

What about EXISTS?

```
SELECT
  x.*
FROM tbl1 AS x
WHERE
  EXISTS
  (
    SELECT
      *
    FROM tbl2 AS y
    WHERE
      y.pk = 123
      AND y.col = x.col
  )
```

What about EXISTS?

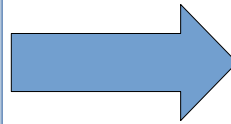
```
SELECT
  x.*
FROM tbl1 AS x
WHERE
  EXISTS
  (
    SELECT
      *
    FROM tbl2 AS y
    WHERE
      y.pk = 123
      AND y.col = x.col
  )
```



**DOES IT
DECORRELATE?**

What about EXISTS?

```
SELECT
  x.*
FROM tbl1 AS x
WHERE
  EXISTS
  (
    SELECT
      *
    FROM tbl2 AS y
    WHERE
      y.pk = 123
      AND y.col = x.col
  )
```



```
SELECT
  x.*
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  y.col = x.col
WHERE
  y.pk = 123
```

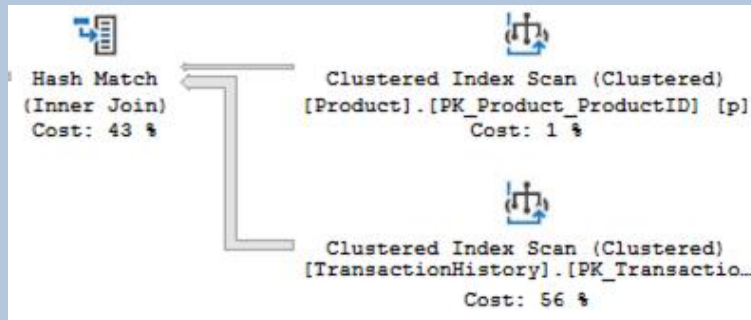
Does SQL Server Agree???

Does SQL Server Agree???

```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Color = 'Red'
  )
```

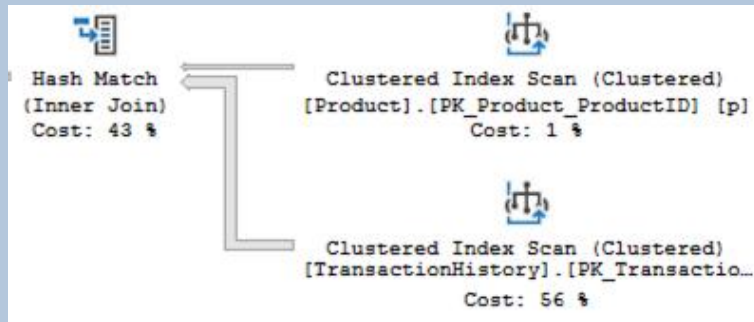
Does SQL Server Agree???

```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Color = 'Red'
  )
```



Does SQL Server Agree???

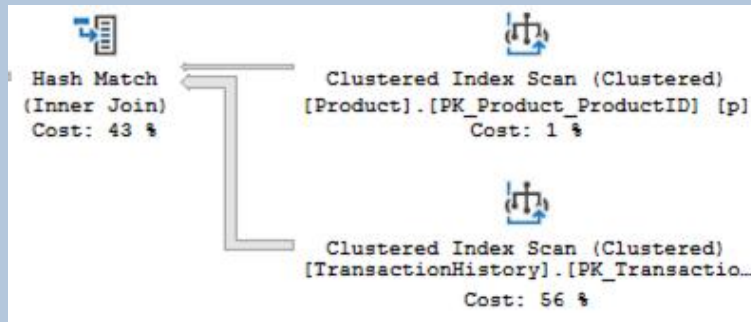
```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Color = 'Red'
  )
```



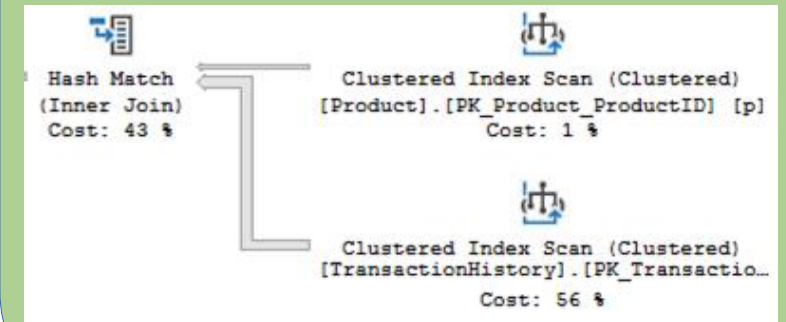
```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```

Does SQL Server Agree???

```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Color = 'Red'
  )
```

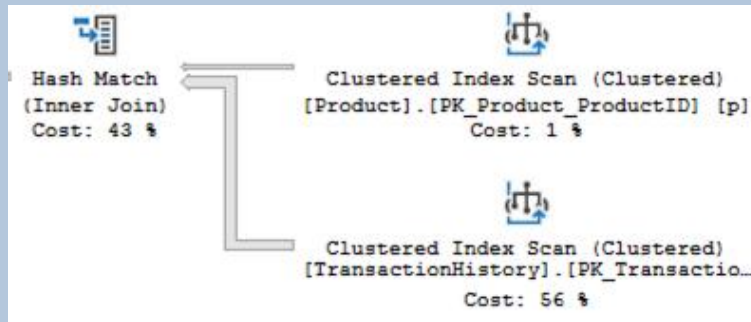


```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```



Does SQL Server Agree???

```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Color = 'Red'
  )
```



```
SELECT
  th.*
FROM TransactionHistory AS th
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
WHERE
  p.Color = 'Red'
```

```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```



Does SQL Server Agree???

```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Color = 'Red'
  )
```



```
SELECT
  th.*
FROM TransactionHistory AS th
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
WHERE
  p.Color = 'Red'
```



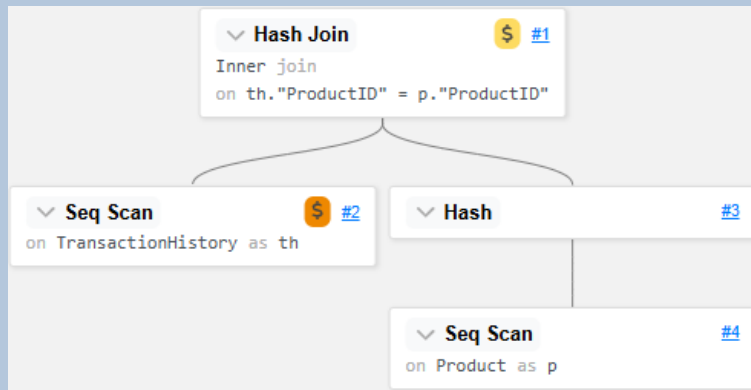
```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```



... and Postgres?

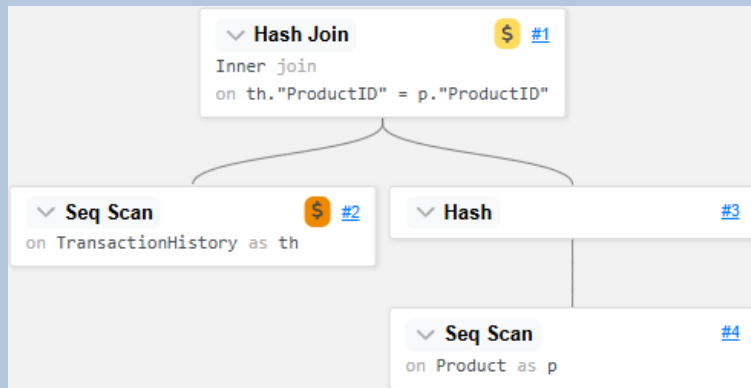
... and Postgres?

```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Color = 'Red'
  )
```

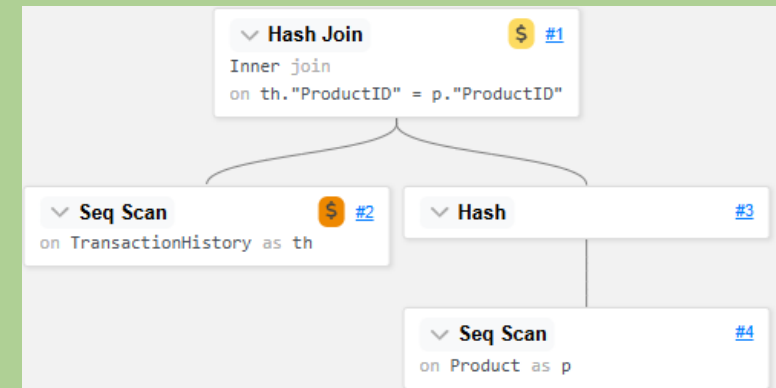


... and Postgres?

```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Color = 'Red'
  )
```

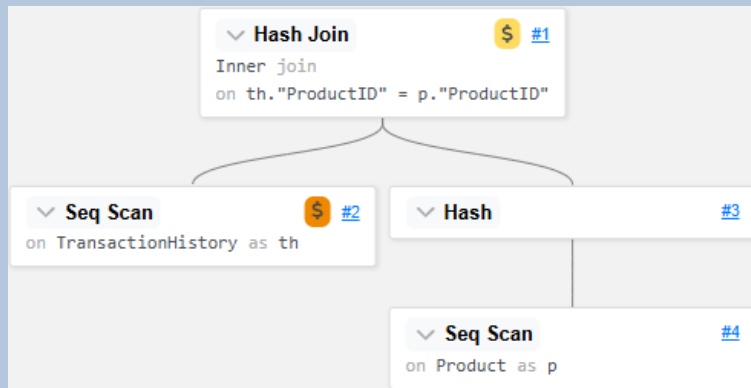


```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```

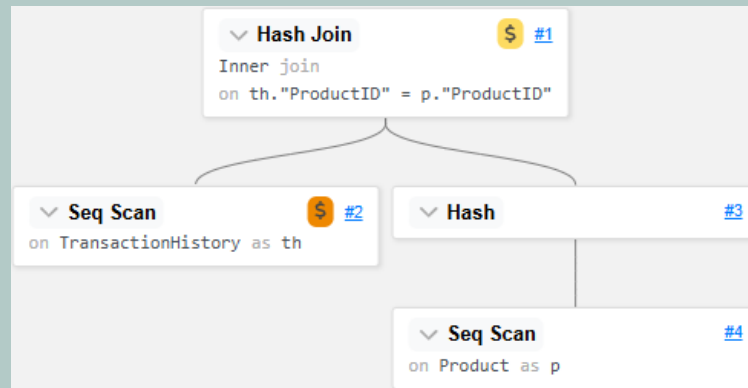


... and Postgres?

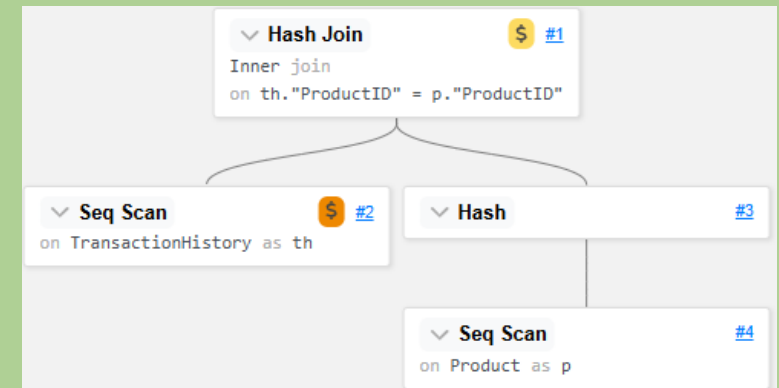
```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Color = 'Red'
  )
```



```
SELECT
  th.*
FROM TransactionHistory AS th
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
WHERE
  p.Color = 'Red'
```



```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```



Does Distinctness Matter?

Does Distinctness Matter?

```
SELECT * INTO #Product FROM Product;  
  
CREATE CLUSTERED INDEX p ON #Product (ProductID);
```

```
CREATE TEMPORARY TABLE tmp_product  
AS SELECT * FROM Product;  
  
CREATE INDEX p ON tmp_product (ProductID);
```

Does Distinctness Matter?

```
SELECT * INTO #Product FROM Product;  
  
CREATE CLUSTERED INDEX p ON #Product (ProductID);
```

```
CREATE TEMPORARY TABLE tmp_product  
AS SELECT * FROM Product;  
  
CREATE INDEX p ON tmp_product (ProductID);
```

TABLE 1

ColA	ColB
1	X
2	Y
2	Z

Does Distinctness Matter?

```
SELECT * INTO #Product FROM Product;
```

```
CREATE CLUSTERED INDEX p ON #Product (ProductID);
```

```
CREATE TEMPORARY TABLE tmp_product  
AS SELECT * FROM Product;
```

```
CREATE INDEX p ON tmp_product (ProductID);
```

TABLE 1

ColA	ColB
1	X
2	Y
2	Z

TABLE 2

ColA	ColC
1	11
2	22
2	33

Does Distinctness Matter?

```
SELECT * INTO #Product FROM Product;
```

```
CREATE CLUSTERED INDEX p ON #Product (ProductID);
```

```
CREATE TEMPORARY TABLE tmp_product  
AS SELECT * FROM Product;
```

```
CREATE INDEX p ON tmp_product (ProductID);
```

TABLE 1		INNER JOIN	TABLE 2	
ColA	ColB		ColA	ColC
1	X		1	11
2	Y		2	22
2	Z		2	33

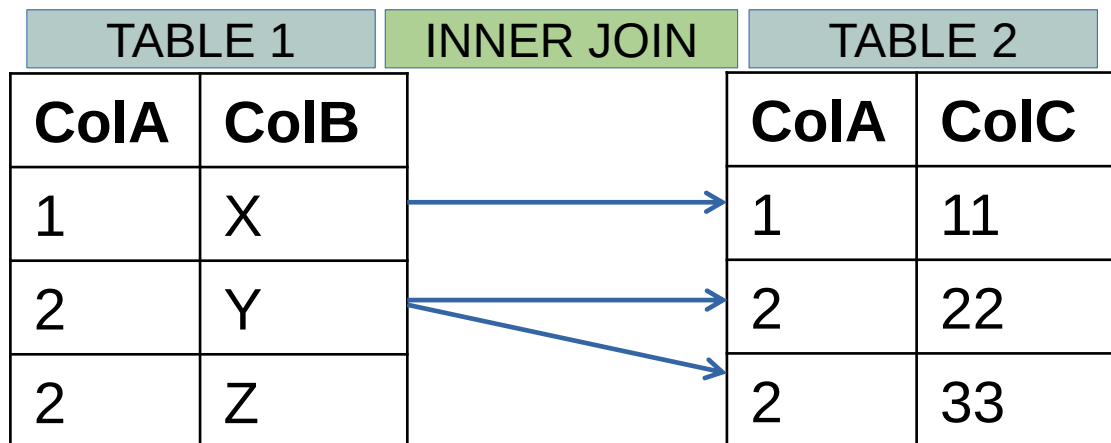
Does Distinctness Matter?

```
SELECT * INTO #Product FROM Product;
```

```
CREATE CLUSTERED INDEX p ON #Product (ProductID);
```

```
CREATE TEMPORARY TABLE tmp_product  
AS SELECT * FROM Product;
```

```
CREATE INDEX p ON tmp_product (ProductID);
```



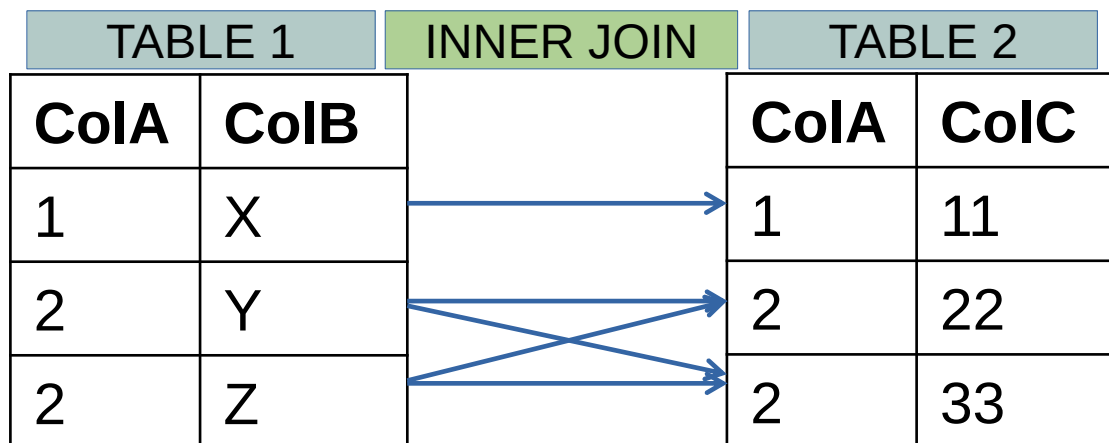
Does Distinctness Matter?

```
SELECT * INTO #Product FROM Product;
```

```
CREATE CLUSTERED INDEX p ON #Product (ProductID);
```

```
CREATE TEMPORARY TABLE tmp_product  
AS SELECT * FROM Product;
```

```
CREATE INDEX p ON tmp_product (ProductID);
```



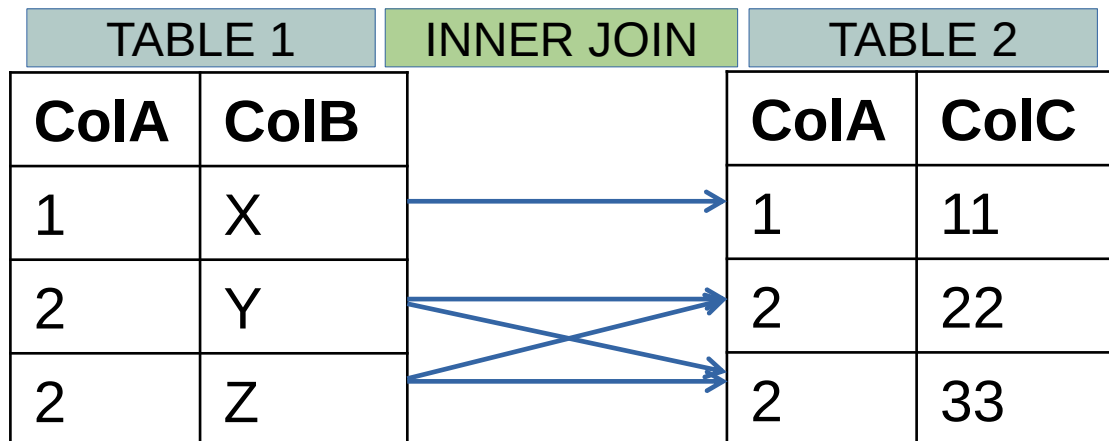
Does Distinctness Matter?

```
SELECT * INTO #Product FROM Product;
```

```
CREATE CLUSTERED INDEX p ON #Product (ProductID);
```

```
CREATE TEMPORARY TABLE tmp_product  
AS SELECT * FROM Product;
```

```
CREATE INDEX p ON tmp_product (ProductID);
```



IS THAT WHAT YOU WANTED?

ColA	ColB	ColC
1	X	11
2	Y	22
2	Z	22
2	Y	33
2	Y	33

SQL Server's New Plans

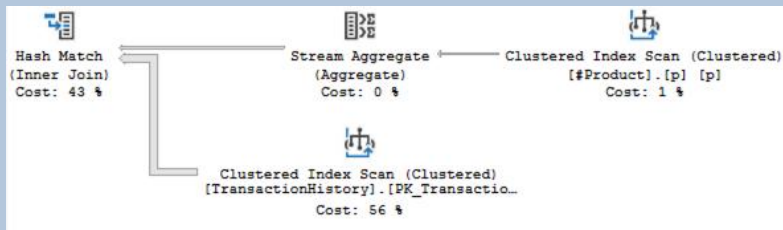
SQL Server's New Plans

```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM #Product AS p
    WHERE
      p.Color = 'Red'
  )
```

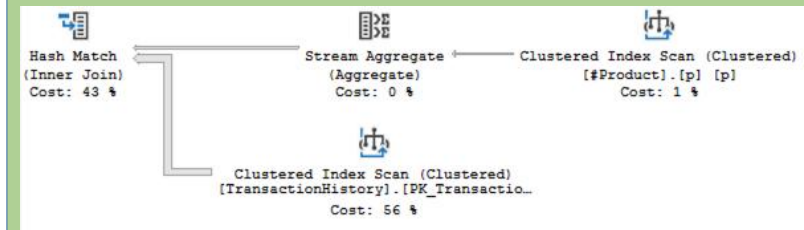
```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM #Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```

SQL Server's New Plans

```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM #Product AS p
    WHERE
      p.Color = 'Red'
  )
```

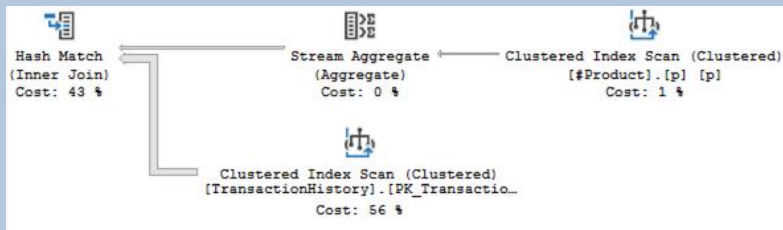


```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM #Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```

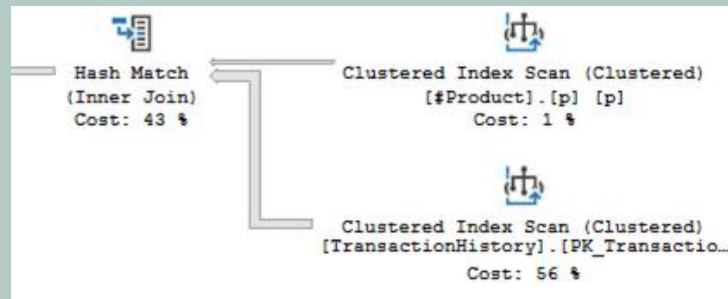


SQL Server's New Plans

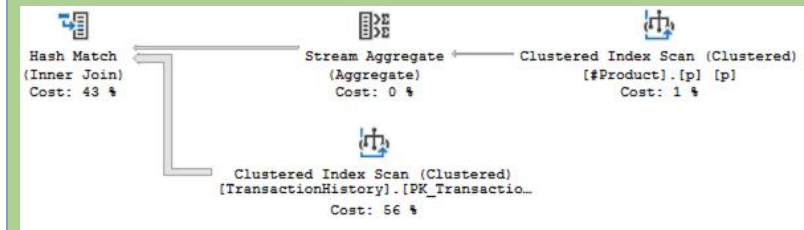
```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM #Product AS p
    WHERE
      p.Color = 'Red'
  )
```



```
SELECT
  th.*
FROM TransactionHistory AS th
INNER JOIN #Product AS p ON
  p.ProductID = th.ProductID
WHERE
  p.Color = 'Red'
```



```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM #Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```



Force DISTINCT Results?

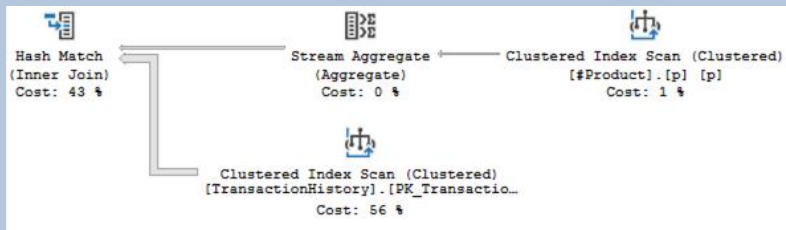
Force DISTINCT Results?

```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM #Product AS p
    WHERE
      p.Color = 'Red'
  )
```

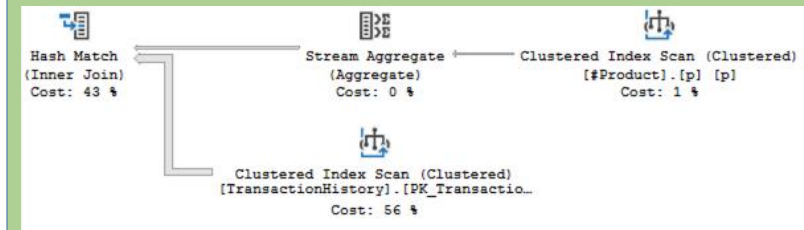
```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM #Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```

Force DISTINCT Results?

```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM #Product AS p
    WHERE
      p.Color = 'Red'
  )
```

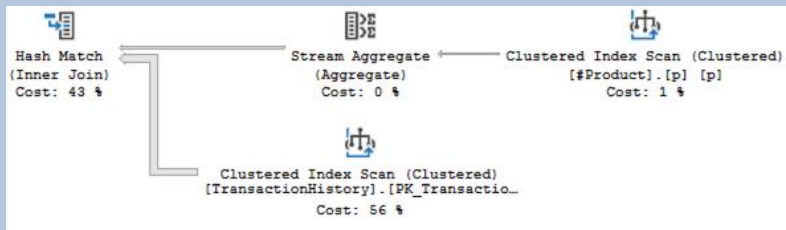


```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM #Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```

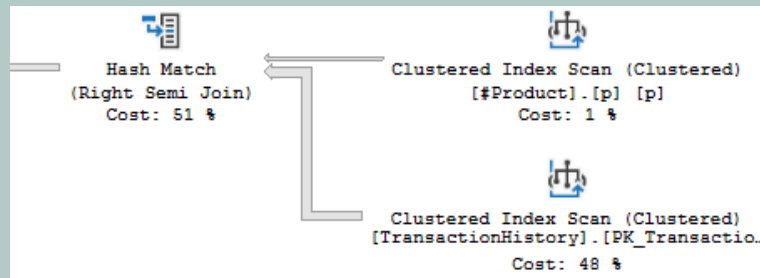


Force DISTINCT Results?

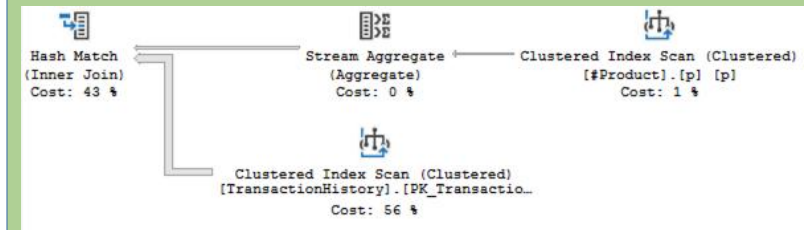
```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM #Product AS p
    WHERE
      p.Color = 'Red'
  )
```



```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
INNER JOIN #Product AS p ON
  p.ProductID = th.ProductID
WHERE
  p.Color = 'Red'
```

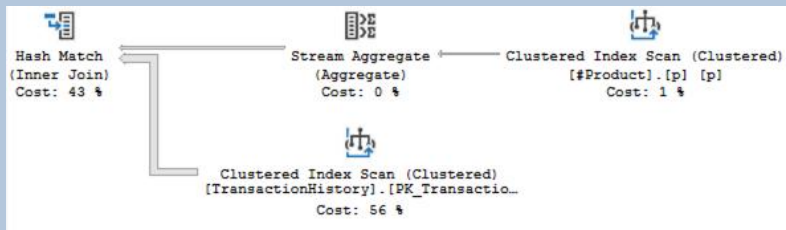


```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM #Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```

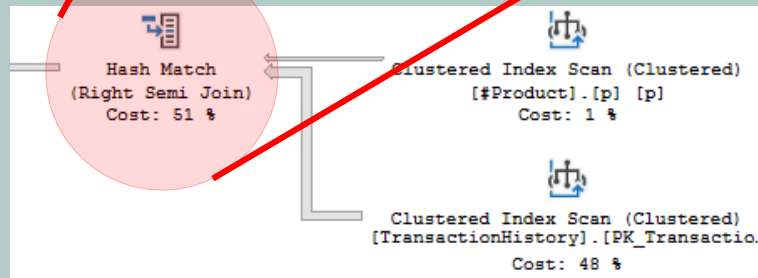
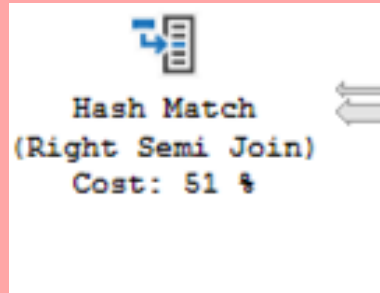


Force DISTINCT Results?

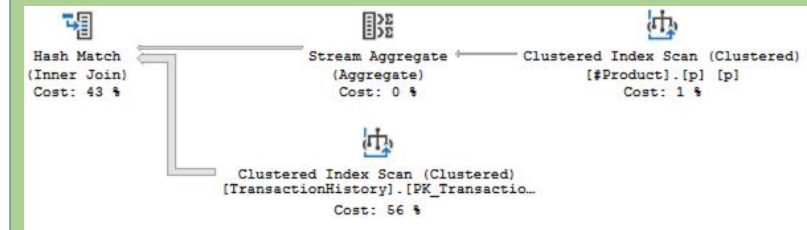
```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM #Product AS p
    WHERE
      p.Color = 'Red'
  )
```



```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
INNER JOIN #Product AS p
  ON p.ProductID = th.ProductID
WHERE
  p.Color = 'Red'
```



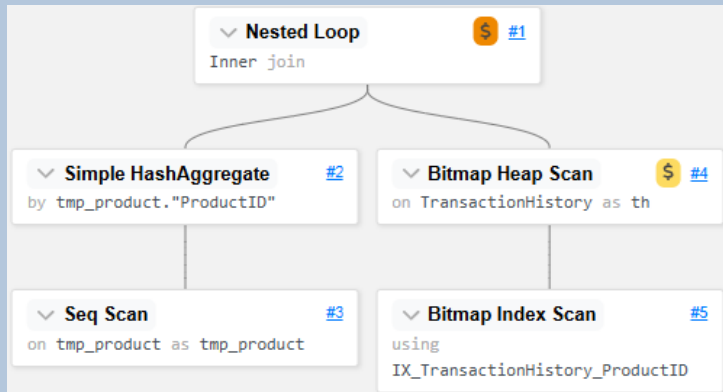
```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM #Product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```



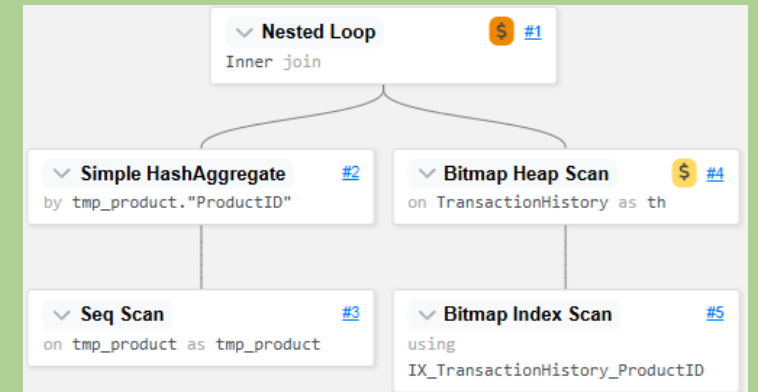
... and Postgres?

... and Postgres?

```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM tmp_product AS p
    WHERE
      p.Color = 'Red'
  )
```

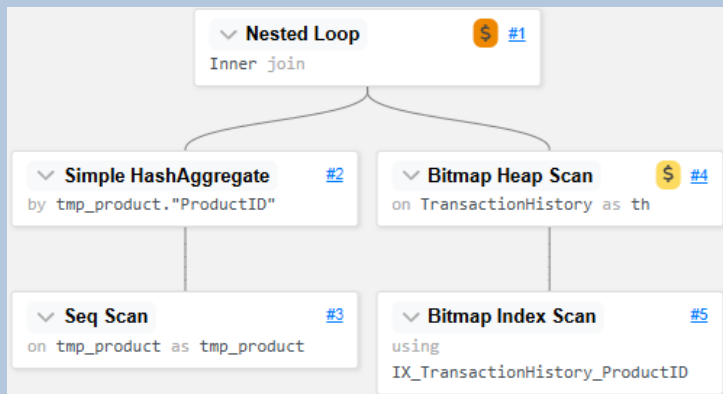


```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM tmp_product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```

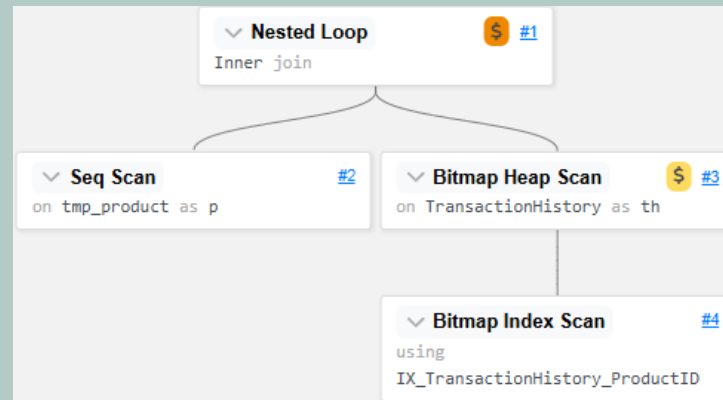


... and Postgres?

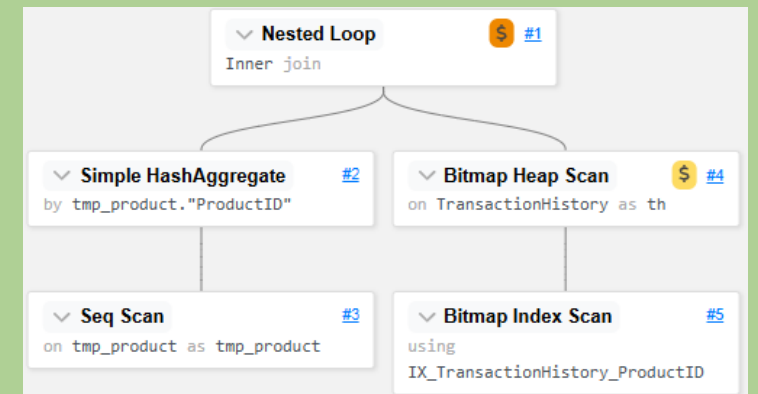
```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM tmp_product AS p
    WHERE
      p.Color = 'Red'
  )
```



```
SELECT
  th.*
FROM TransactionHistory AS th
INNER JOIN tmp_product AS p ON
  p.ProductID = th.ProductID
WHERE
  p.Color = 'Red'
```



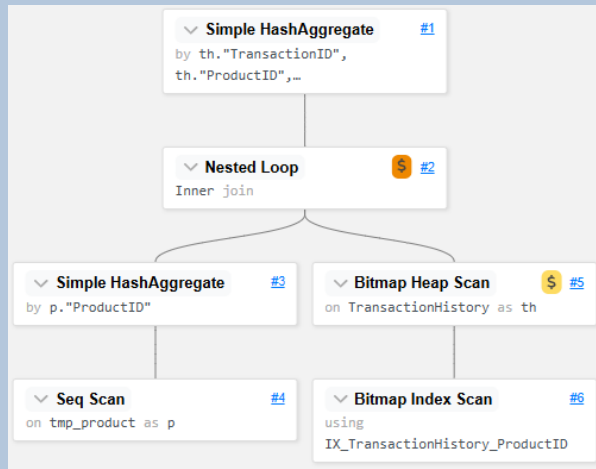
```
SELECT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM tmp_product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```



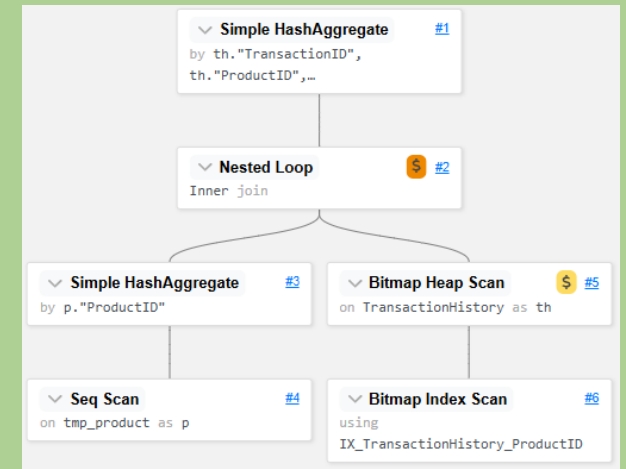
Postgres DISTINCT

Postgres DISTINCT

```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM tmp_product AS p
    WHERE
      p.Color = 'Red'
  )
```

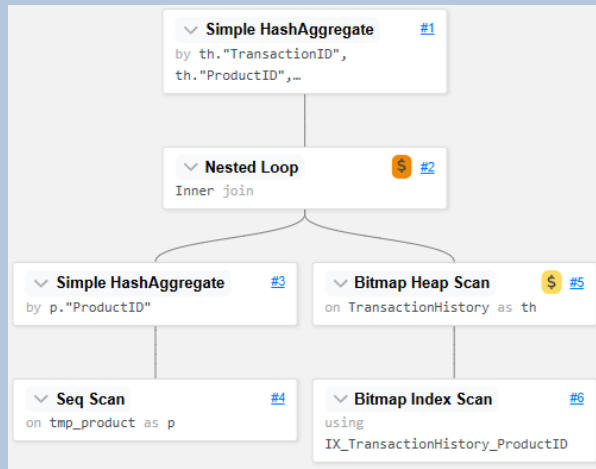


```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM tmp_product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```

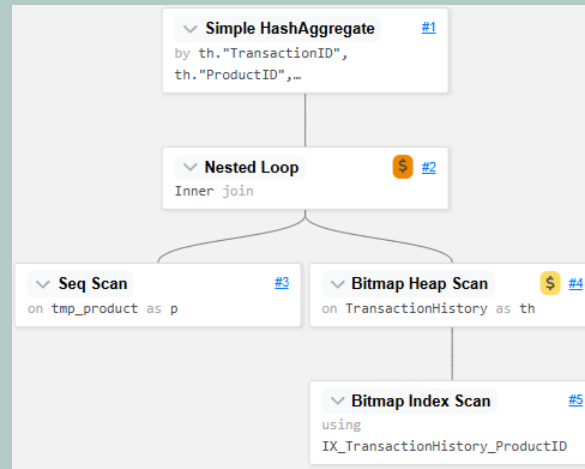


Postgres DISTINCT

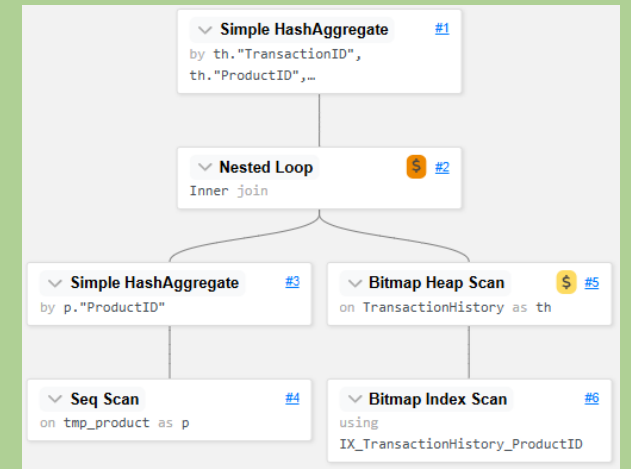
```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM tmp_product AS p
    WHERE
      p.Color = 'Red'
  )
```



```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
INNER JOIN tmp_product AS p ON
  p.ProductID = th.ProductID
WHERE
  p.Color = 'Red'
```

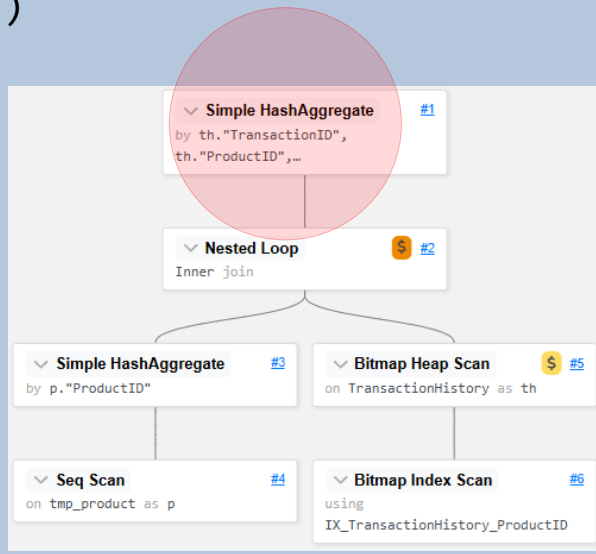


```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM tmp_product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```

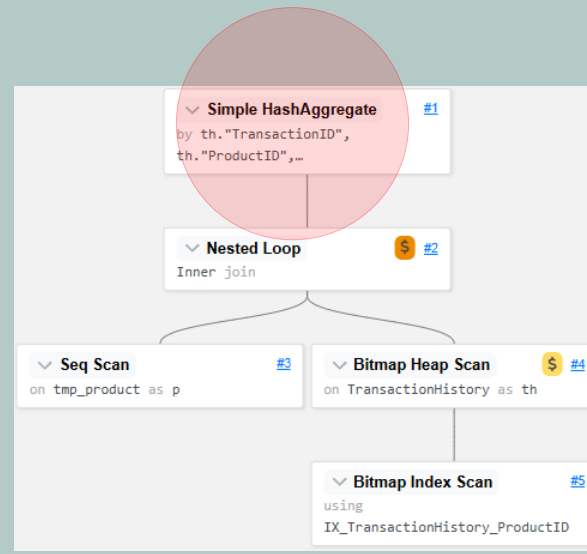


Postgres DISTINCT...Disappoints

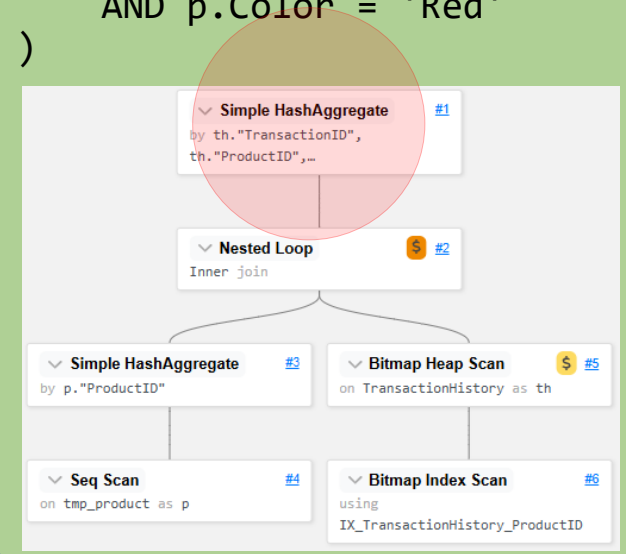
```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM tmp_product AS p
    WHERE
      p.Color = 'Red'
  )
```



```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
INNER JOIN tmp_product AS p ON
  p.ProductID = th.ProductID
WHERE
  p.Color = 'Red'
```

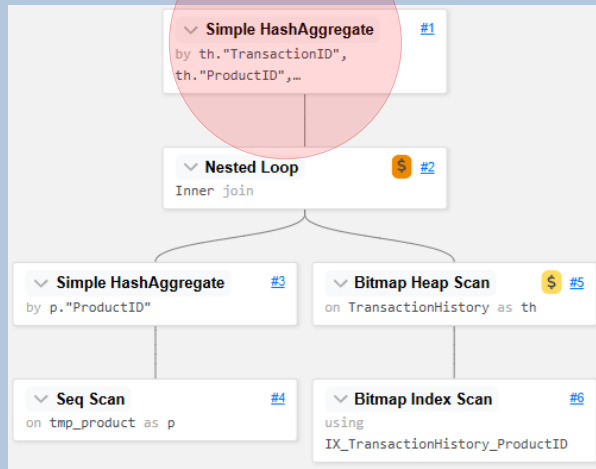


```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  EXISTS
  (
    SELECT
      *
    FROM tmp_product AS p
    WHERE
      p.ProductID = th.ProductID
      AND p.Color = 'Red'
  )
```

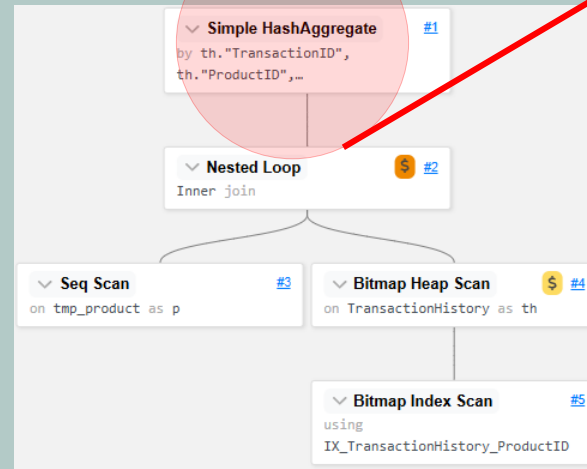


Postgres DISTINCT...Disappoints

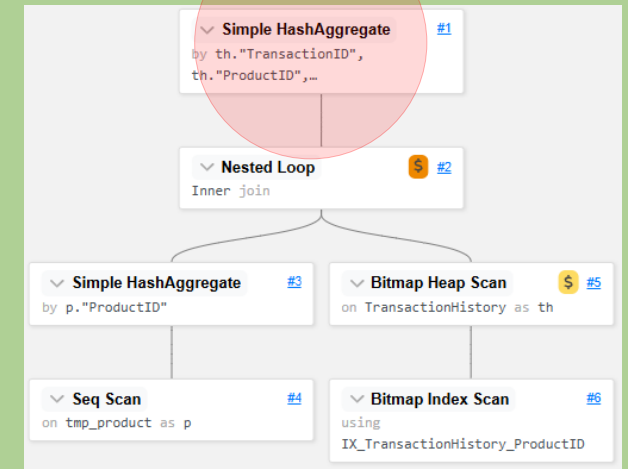
```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
WHERE
  th.ProductID IN
  (
    SELECT
      p.ProductID
    FROM tmp_product AS p
    WHERE
      p.Color = 'Red'
  )
```



```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
INNER JOIN tmp_product AS p
  ON p.ProductID = th.ProductID
WHERE
  p.Color = 'Red'
```



```
SELECT DISTINCT
  th.*
FROM TransactionHistory AS th
INNER JOIN tmp_product AS p
  ON p.ProductID = th.ProductID
WHERE
  p.ProductID = th.ProductID
  AND p.Color = 'Red'
```



The Truth About Semijoins

The Truth About Semijoins

**Use EXISTS
instead of IN**

~~“It’s much faster.”~~
...if you like to
read EXISTS
better than IN

The Truth About Semijoins

**Use EXISTS
instead of IN**

~~“It’s much faster.”~~
...if you like to
read EXISTS
better than IN

**Don’t use
EXISTS or IN**

~~“INNER JOIN
scales better”~~
...INNER JOIN
gives you free
extra rows!

in wisdom : CTEs

in wisdom : CTEs

**Use CTEs
instead of
subqueries**

“For better
performance.”

in wisdom : CTEs

**Use CTEs
instead of
subqueries**

“For better
performance.”

**Don't use too
many CTEs**

“The optimizer will
get confused.”

SQL Server: CTE, Subquery, 

SQL Server: CTE, Subquery,

```
SELECT
  COUNT(*)
FROM TransactionHistory AS th
WHERE
  ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Name = 'Cable Lock'
  )
```

```
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN
  (
    SELECT
      *
    FROM Product AS p0
    WHERE
      p0.Name = 'Cable Lock'
  ) AS p ON
  p.ProductID = th.ProductID
```

SQL Server: CTE, Subquery,

```
SELECT
  COUNT(*)
FROM TransactionHistory AS th
WHERE
  ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Name = 'Cable Lock'
  )
```

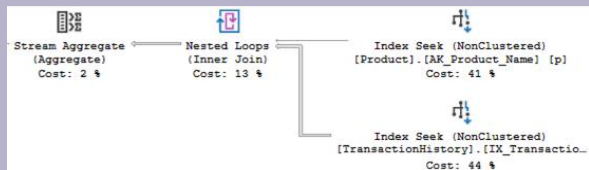
```
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN
  (
    SELECT
      *
    FROM Product AS p0
    WHERE
      p0.Name = 'Cable Lock'
  ) AS p ON
  p.ProductID = th.ProductID
```

```
WITH p AS
(
  SELECT
    *
  FROM Product AS p0
  WHERE
    p0.Name = 'Cable Lock'
)
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
  p.ProductID = th.ProductID
```

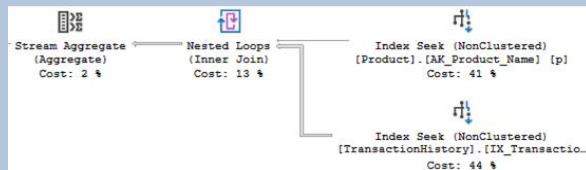
```
WITH p AS
(
  SELECT
    *
  FROM Product
)
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
  p.ProductID = th.ProductID
WHERE
  p.Name = 'Cable Lock'
```

SQL Server: CTE, Subquery,

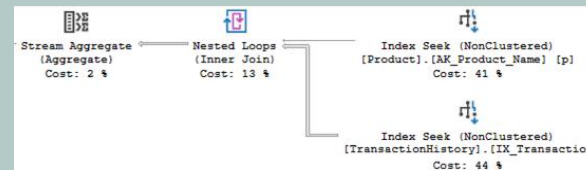
```
SELECT
  COUNT(*)
FROM TransactionHistory AS th
WHERE
  ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Name = 'Cable Lock'
  )
```



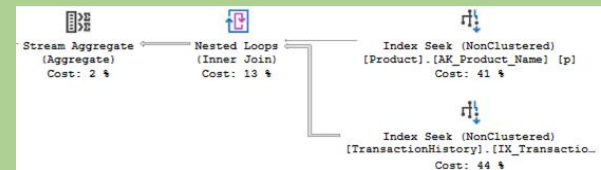
```
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN
  (
    SELECT
      *
    FROM Product AS p0
    WHERE
      p0.Name = 'Cable Lock'
  ) AS p ON
  p.ProductID = th.ProductID
```



```
WITH p AS
(
  SELECT
    *
  FROM Product AS p0
  WHERE
    p0.Name = 'Cable Lock'
)
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
  p.ProductID = th.ProductID
```



```
WITH p AS
(
  SELECT
    *
  FROM Product
)
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
  p.ProductID = th.ProductID
WHERE
  p.Name = 'Cable Lock'
```



Postgres:  ? 🙄

Postgres: ?

```
SELECT
  COUNT(*)
FROM TransactionHistory AS th
WHERE
  ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Name = 'Cable Lock'
  )
```

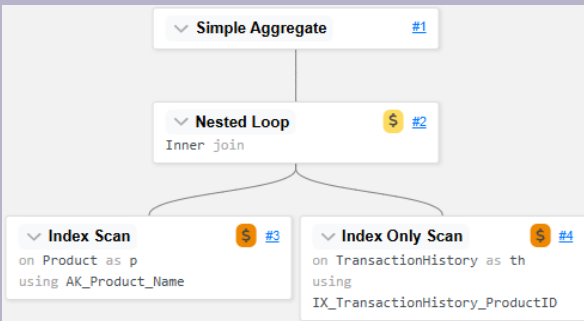
```
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN
  (
    SELECT
      *
    FROM Product AS p0
    WHERE
      p0.Name = 'Cable Lock'
  ) AS p ON
  p.ProductID = th.ProductID
```

```
WITH p AS
(
  SELECT
    *
  FROM Product AS p0
  WHERE
    p0.Name = 'Cable Lock'
)
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
  p.ProductID = th.ProductID
```

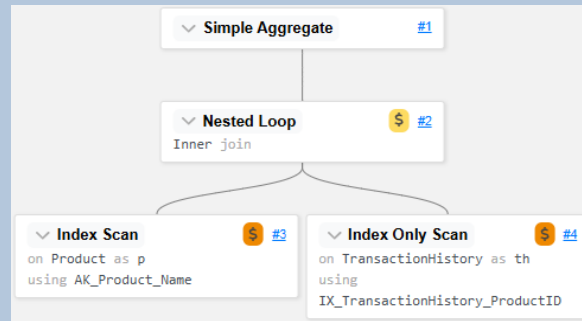
```
WITH p AS
(
  SELECT
    *
  FROM Product
)
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
  p.ProductID = th.ProductID
WHERE
  p.Name = 'Cable Lock'
```

Postgres: ?

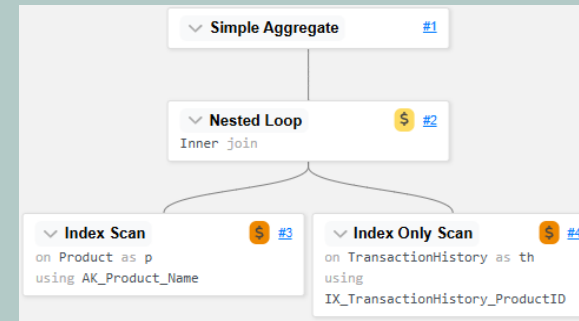
```
SELECT
  COUNT(*)
FROM TransactionHistory AS th
WHERE
  ProductID IN
  (
    SELECT
      p.ProductID
    FROM Product AS p
    WHERE
      p.Name = 'Cable Lock'
  )
```



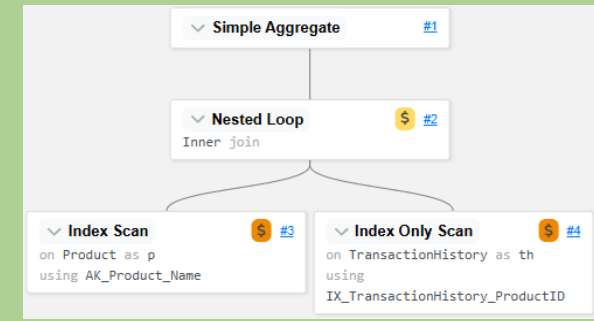
```
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN
  (
    SELECT
      *
    FROM Product AS p0
    WHERE
      p0.Name = 'Cable Lock'
  ) AS p ON
  p.ProductID = th.ProductID
```



```
WITH p AS
(
  SELECT
    *
  FROM Product AS p0
  WHERE
    p0.Name = 'Cable Lock'
)
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
  p.ProductID = th.ProductID
```



```
WITH p AS
(
  SELECT
    *
  FROM Product
)
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
  p.ProductID = th.ProductID
WHERE
  p.Name = 'Cable Lock'
```



Does SQL Server Get Confused?

Does SQL Server Get Confused?

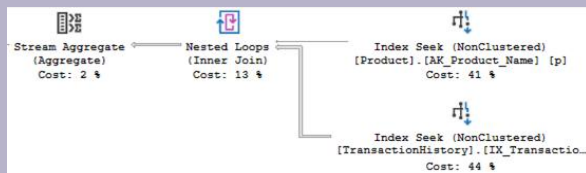
1 CTE

```
WITH p AS  
(  
    SELECT  
        *  
    FROM Product  
)  
SELECT  
    COUNT(*)  
FROM TransactionHistory AS th  
INNER JOIN p ON  
    p.ProductID = th.ProductID  
WHERE  
    p.Name = 'Cable Lock'
```


Does SQL Server Get Confused?

1 CTE

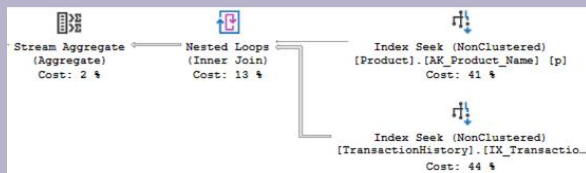
```
WITH p AS  
(  
    SELECT  
        *  
    FROM Product  
)  
SELECT  
    COUNT(*)  
FROM TransactionHistory AS th  
INNER JOIN p ON  
    p.ProductID = th.ProductID  
WHERE  
    p.Name = 'Cable Lock'
```



Does SQL Server Get Confused?

1 CTE

```
WITH p AS
(
    SELECT
        *
    FROM Product
)
SELECT
    COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
    p.ProductID = th.ProductID
WHERE
    p.Name = 'Cable Lock'
```



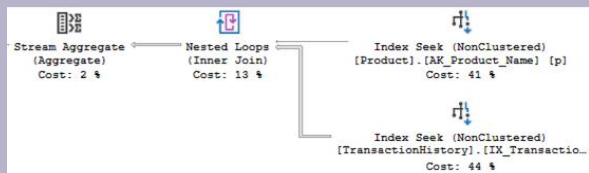
10 CTEs

```
WITH p0 AS
(
    SELECT
        *
    FROM Production.Product
)
,p1 AS (SELECT * FROM p0)
,p2 AS (SELECT * FROM p1)
,p3 AS (SELECT * FROM p2)
,p4 AS (SELECT * FROM p3)
,p5 AS (SELECT * FROM p4)
,p6 AS (SELECT * FROM p5)
,p7 AS (SELECT * FROM p6)
,p8 AS (SELECT * FROM p7)
,p9 AS (SELECT * FROM p8)
SELECT
    COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p9 ON
    p9.ProductID = th.ProductID
WHERE
    p9.Name = 'Cable Lock'
```

Does SQL Server Get Confused?

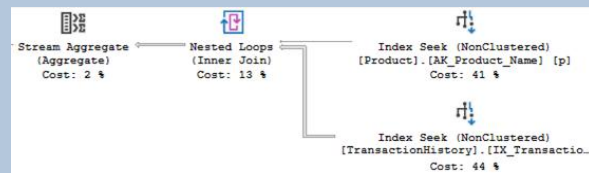
1 CTE

```
WITH p AS
(
    SELECT
        *
    FROM Product
)
SELECT
    COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
    p.ProductID = th.ProductID
WHERE
    p.Name = 'Cable Lock'
```



10 CTEs

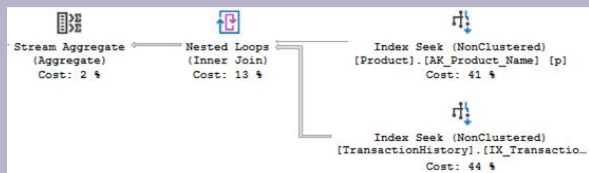
```
WITH p0 AS
(
    SELECT
        *
    FROM Production.Product
)
,p1 AS (SELECT * FROM p0)
,p2 AS (SELECT * FROM p1)
,p3 AS (SELECT * FROM p2)
,p4 AS (SELECT * FROM p3)
,p5 AS (SELECT * FROM p4)
,p6 AS (SELECT * FROM p5)
,p7 AS (SELECT * FROM p6)
,p8 AS (SELECT * FROM p7)
,p9 AS (SELECT * FROM p8)
SELECT
    COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p9 ON
    p9.ProductID = th.ProductID
WHERE
    p9.Name = 'Cable Lock'
```



Does SQL Server Get Confused?

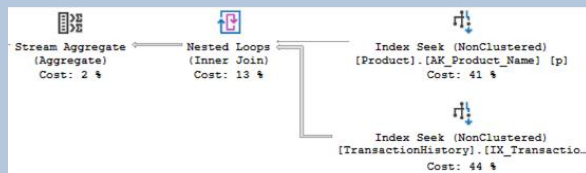
1 CTE

```
WITH p AS
(
    SELECT
        *
    FROM Product
)
SELECT
    COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
    p.ProductID = th.ProductID
WHERE
    p.Name = 'Cable Lock'
```



10 CTEs

```
WITH p0 AS
(
    SELECT
        *
    FROM Production.Product
)
,p1 AS (SELECT * FROM p0)
,p2 AS (SELECT * FROM p1)
,p3 AS (SELECT * FROM p2)
,p4 AS (SELECT * FROM p3)
,p5 AS (SELECT * FROM p4)
,p6 AS (SELECT * FROM p5)
,p7 AS (SELECT * FROM p6)
,p8 AS (SELECT * FROM p7)
,p9 AS (SELECT * FROM p8)
SELECT
    COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p9 ON
    p9.ProductID = th.ProductID
WHERE
    p9.Name = 'Cable Lock'
```



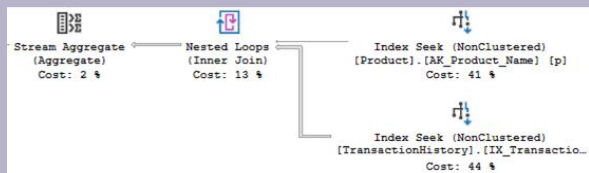
100 CTEs

```
...
,p72 AS (SELECT * FROM p71)
,p73 AS (SELECT * FROM p72)
,p74 AS (SELECT * FROM p73)
,p75 AS (SELECT * FROM p74)
,p76 AS (SELECT * FROM p75)
,p77 AS (SELECT * FROM p76)
,p78 AS (SELECT * FROM p77)
,p79 AS (SELECT * FROM p78)
,p80 AS (SELECT * FROM p79)
,p81 AS (SELECT * FROM p80)
,p82 AS (SELECT * FROM p81)
,p83 AS (SELECT * FROM p82)
,p84 AS (SELECT * FROM p83)
,p85 AS (SELECT * FROM p84)
,p86 AS (SELECT * FROM p85)
,p87 AS (SELECT * FROM p86)
,p88 AS (SELECT * FROM p87)
,p89 AS (SELECT * FROM p88)
,p90 AS (SELECT * FROM p89)
,p91 AS (SELECT * FROM p90)
,p92 AS (SELECT * FROM p91)
,p93 AS (SELECT * FROM p92)
,p94 AS (SELECT * FROM p93)
,p95 AS (SELECT * FROM p94)
,p96 AS (SELECT * FROM p95)
,p97 AS (SELECT * FROM p96)
,p98 AS (SELECT * FROM p97)
,p99 AS (SELECT * FROM p98)
SELECT
    COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p99 ON
    p99.ProductID = th.ProductID
WHERE
    p99.Name = 'Cable Lock'
```

Does SQL Server Get Confused?

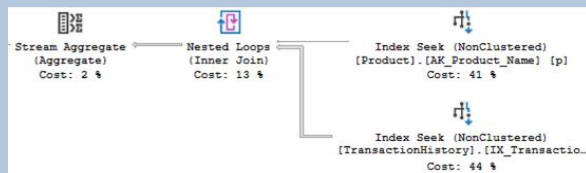
1 CTE

```
WITH p AS
(
    SELECT
        *
    FROM Product
)
SELECT
    COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
    p.ProductID = th.ProductID
WHERE
    p.Name = 'Cable Lock'
```



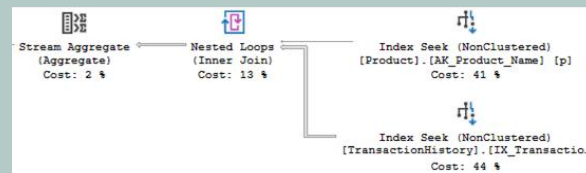
10 CTEs

```
WITH p0 AS
(
    SELECT
        *
    FROM Production.Product
)
,p1 AS (SELECT * FROM p0)
,p2 AS (SELECT * FROM p1)
,p3 AS (SELECT * FROM p2)
,p4 AS (SELECT * FROM p3)
,p5 AS (SELECT * FROM p4)
,p6 AS (SELECT * FROM p5)
,p7 AS (SELECT * FROM p6)
,p8 AS (SELECT * FROM p7)
,p9 AS (SELECT * FROM p8)
SELECT
    COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p9 ON
    p9.ProductID = th.ProductID
WHERE
    p9.Name = 'Cable Lock'
```



100 CTEs

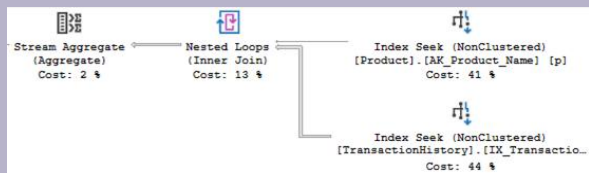
```
...
,p72 AS (SELECT * FROM p71)
,p73 AS (SELECT * FROM p72)
,p74 AS (SELECT * FROM p73)
,p75 AS (SELECT * FROM p74)
,p76 AS (SELECT * FROM p75)
,p77 AS (SELECT * FROM p76)
,p78 AS (SELECT * FROM p77)
,p79 AS (SELECT * FROM p78)
,p80 AS (SELECT * FROM p79)
,p81 AS (SELECT * FROM p80)
,p82 AS (SELECT * FROM p81)
,p83 AS (SELECT * FROM p82)
,p84 AS (SELECT * FROM p83)
,p85 AS (SELECT * FROM p84)
,p86 AS (SELECT * FROM p85)
,p87 AS (SELECT * FROM p86)
,p88 AS (SELECT * FROM p87)
,p89 AS (SELECT * FROM p88)
,p90 AS (SELECT * FROM p89)
,p91 AS (SELECT * FROM p90)
,p92 AS (SELECT * FROM p91)
,p93 AS (SELECT * FROM p92)
,p94 AS (SELECT * FROM p93)
,p95 AS (SELECT * FROM p94)
,p96 AS (SELECT * FROM p95)
,p97 AS (SELECT * FROM p96)
,p98 AS (SELECT * FROM p97)
,p99 AS (SELECT * FROM p98)
SELECT
    COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p99 ON
    p99.ProductID = th.ProductID
WHERE
    p99.Name = 'Cable Lock'
```



Does SQL Server Get Confused?

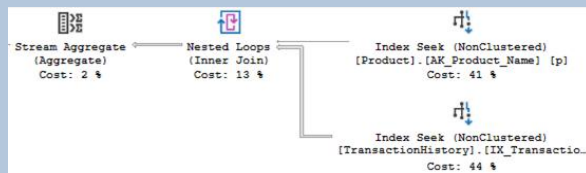
1 CTE

```
WITH p AS
(
    SELECT
        *
    FROM Product
)
SELECT
    COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
    p.ProductID = th.ProductID
WHERE
    p.Name = 'Cable Lock'
```



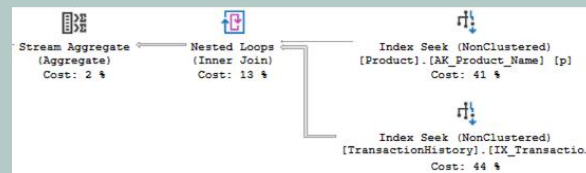
10 CTEs

```
WITH p0 AS
(
    SELECT
        *
    FROM Production.Product
)
,p1 AS (SELECT * FROM p0)
,p2 AS (SELECT * FROM p1)
,p3 AS (SELECT * FROM p2)
,p4 AS (SELECT * FROM p3)
,p5 AS (SELECT * FROM p4)
,p6 AS (SELECT * FROM p5)
,p7 AS (SELECT * FROM p6)
,p8 AS (SELECT * FROM p7)
,p9 AS (SELECT * FROM p8)
SELECT
    COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p9 ON
    p9.ProductID = th.ProductID
WHERE
    p9.Name = 'Cable Lock'
```



100 CTEs

```
...
,p72 AS (SELECT * FROM p71)
,p73 AS (SELECT * FROM p72)
,p74 AS (SELECT * FROM p73)
,p75 AS (SELECT * FROM p74)
,p76 AS (SELECT * FROM p75)
,p77 AS (SELECT * FROM p76)
,p78 AS (SELECT * FROM p77)
,p79 AS (SELECT * FROM p78)
,p80 AS (SELECT * FROM p79)
,p81 AS (SELECT * FROM p80)
,p82 AS (SELECT * FROM p81)
,p83 AS (SELECT * FROM p82)
,p84 AS (SELECT * FROM p83)
,p85 AS (SELECT * FROM p84)
,p86 AS (SELECT * FROM p85)
,p87 AS (SELECT * FROM p86)
,p88 AS (SELECT * FROM p87)
,p89 AS (SELECT * FROM p88)
,p90 AS (SELECT * FROM p89)
,p91 AS (SELECT * FROM p90)
,p92 AS (SELECT * FROM p91)
,p93 AS (SELECT * FROM p92)
,p94 AS (SELECT * FROM p93)
,p95 AS (SELECT * FROM p94)
,p96 AS (SELECT * FROM p95)
,p97 AS (SELECT * FROM p96)
,p98 AS (SELECT * FROM p97)
,p99 AS (SELECT * FROM p98)
SELECT
    COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p99 ON
    p99.ProductID = th.ProductID
WHERE
    p99.Name = 'Cable Lock'
```



1000 CTEs!

```
...
,p990 AS (SELECT * FROM p989)
,p991 AS (SELECT * FROM p990)
,p992 AS (SELECT * FROM p991)
,p993 AS (SELECT * FROM p992)
,p994 AS (SELECT * FROM p993)
,p995 AS (SELECT * FROM p994)
,p996 AS (SELECT * FROM p995)
,p997 AS (SELECT * FROM p996)
,p998 AS (SELECT * FROM p997)
,p999 AS (SELECT * FROM p998)
SELECT
    COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p999 ON
    p999.ProductID = th.ProductID
WHERE
    p999.Name = 'Cable Lock'
```

Does SQL Server Get Confused?

1 CTE

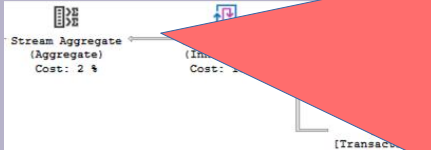
```
WITH p AS  
(  
    SELECT  
        *  
    FROM Product  
)  
SELECT  
    COUNT(*)  
FROM TransactionHistory  
th  
INNER JOIN p ON  
    p.ProductID = th.ProductID  
WHERE  
    p.Name = 'Cable Lock'
```

10 CTEs

```
WITH p0 AS  
(  
    SELECT  
        *  
    FROM Production.  
)  
,p1 AS (SELECT * FROM  
,p2 AS (SELECT * FROM  
,p3 AS (SELECT * FROM
```

100 CTEs!

Msg 701, Level 17, State 123, Line 5
There is insufficient system memory in resource pool 'default' to run this query.

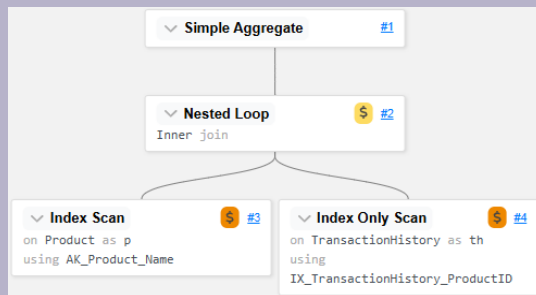


Can Postgres Handle It???

Can Postgres Handle It???

1 CTE

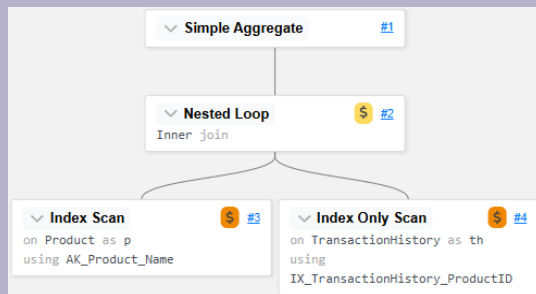
```
WITH p AS  
(  
  SELECT  
    *  
  FROM Product  
)  
SELECT  
  COUNT(*)  
FROM TransactionHistory AS th  
INNER JOIN p ON  
  p.ProductID = th.ProductID  
WHERE  
  p.Name = 'Cable Lock'
```



Can Postgres Handle It???

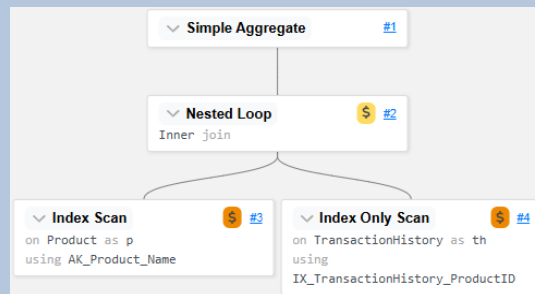
1 CTE

```
WITH p AS
(
  SELECT
    *
  FROM Product
)
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
  p.ProductID = th.ProductID
WHERE
  p.Name = 'Cable Lock'
```



10 CTEs

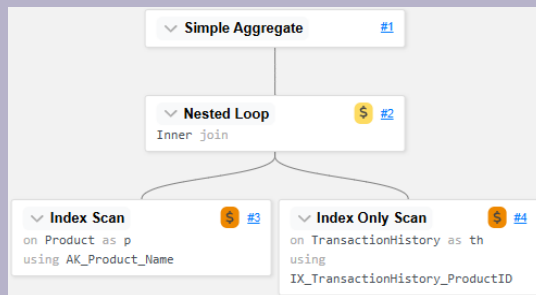
```
WITH p0 AS
(
  SELECT
    *
  FROM Production.Product
)
,p1 AS (SELECT * FROM p0)
,p2 AS (SELECT * FROM p1)
,p3 AS (SELECT * FROM p2)
,p4 AS (SELECT * FROM p3)
,p5 AS (SELECT * FROM p4)
,p6 AS (SELECT * FROM p5)
,p7 AS (SELECT * FROM p6)
,p8 AS (SELECT * FROM p7)
,p9 AS (SELECT * FROM p8)
SELECT
  COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p9 ON
  p9.ProductID = th.ProductID
WHERE
  p9.Name = 'Cable Lock'
```



Can Postgres Handle It???

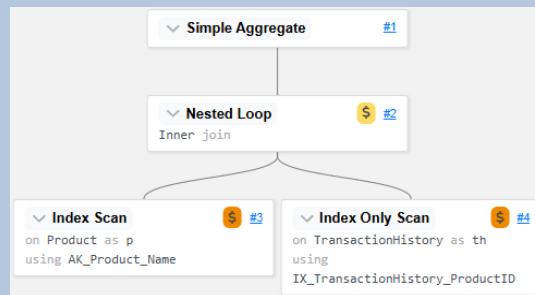
1 CTE

```
WITH p AS
(
  SELECT
    *
  FROM Product
)
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
  p.ProductID = th.ProductID
WHERE
  p.Name = 'Cable Lock'
```



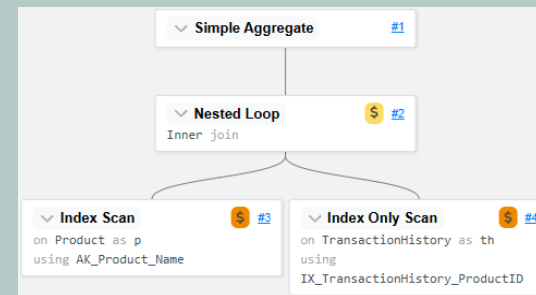
10 CTEs

```
WITH p0 AS
(
  SELECT
    *
  FROM Production.Product
)
,p1 AS (SELECT * FROM p0)
,p2 AS (SELECT * FROM p1)
,p3 AS (SELECT * FROM p2)
,p4 AS (SELECT * FROM p3)
,p5 AS (SELECT * FROM p4)
,p6 AS (SELECT * FROM p5)
,p7 AS (SELECT * FROM p6)
,p8 AS (SELECT * FROM p7)
,p9 AS (SELECT * FROM p8)
SELECT
  COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p9 ON
  p9.ProductID = th.ProductID
WHERE
  p9.Name = 'Cable Lock'
```



100 CTEs

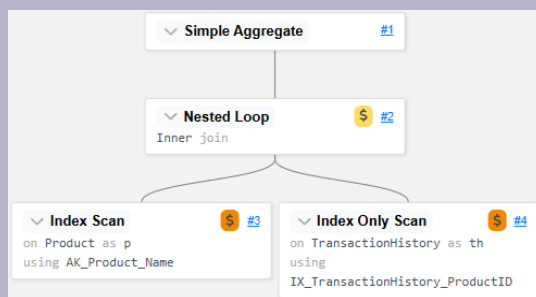
```
...
,p72 AS (SELECT * FROM p71)
,p73 AS (SELECT * FROM p72)
,p74 AS (SELECT * FROM p73)
,p75 AS (SELECT * FROM p74)
,p76 AS (SELECT * FROM p75)
,p77 AS (SELECT * FROM p76)
,p78 AS (SELECT * FROM p77)
,p79 AS (SELECT * FROM p78)
,p80 AS (SELECT * FROM p79)
,p81 AS (SELECT * FROM p80)
,p82 AS (SELECT * FROM p81)
,p83 AS (SELECT * FROM p82)
,p84 AS (SELECT * FROM p83)
,p85 AS (SELECT * FROM p84)
,p86 AS (SELECT * FROM p85)
,p87 AS (SELECT * FROM p86)
,p88 AS (SELECT * FROM p87)
,p89 AS (SELECT * FROM p88)
,p90 AS (SELECT * FROM p89)
,p91 AS (SELECT * FROM p90)
,p92 AS (SELECT * FROM p91)
,p93 AS (SELECT * FROM p92)
,p94 AS (SELECT * FROM p93)
,p95 AS (SELECT * FROM p94)
,p96 AS (SELECT * FROM p95)
,p97 AS (SELECT * FROM p96)
,p98 AS (SELECT * FROM p97)
,p99 AS (SELECT * FROM p98)
SELECT
  COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p99 ON
  p99.ProductID = th.ProductID
WHERE
  p99.Name = 'Cable Lock'
```



Can Postgres Handle It???

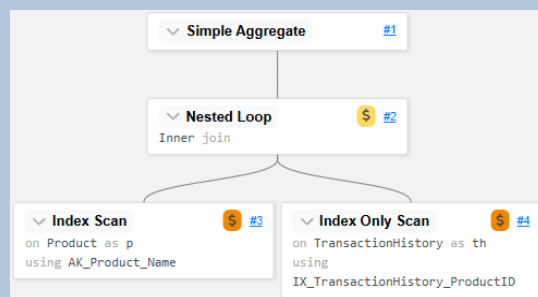
1 CTE

```
WITH p AS
(
  SELECT
    *
  FROM Product
)
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
  p.ProductID = th.ProductID
WHERE
  p.Name = 'Cable Lock'
```



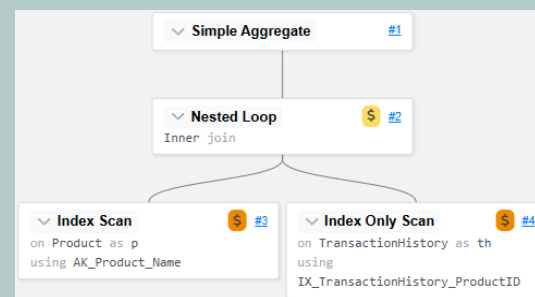
10 CTEs

```
WITH p0 AS
(
  SELECT
    *
  FROM Production.Product
),
p1 AS (SELECT * FROM p0),
p2 AS (SELECT * FROM p1),
p3 AS (SELECT * FROM p2),
p4 AS (SELECT * FROM p3),
p5 AS (SELECT * FROM p4),
p6 AS (SELECT * FROM p5),
p7 AS (SELECT * FROM p6),
p8 AS (SELECT * FROM p7),
p9 AS (SELECT * FROM p8)
SELECT
  COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p9 ON
  p9.ProductID = th.ProductID
WHERE
  p9.Name = 'Cable Lock'
```



100 CTEs

```
...
,p72 AS (SELECT * FROM p71),
,p73 AS (SELECT * FROM p72),
,p74 AS (SELECT * FROM p73),
,p75 AS (SELECT * FROM p74),
,p76 AS (SELECT * FROM p75),
,p77 AS (SELECT * FROM p76),
,p78 AS (SELECT * FROM p77),
,p79 AS (SELECT * FROM p78),
,p80 AS (SELECT * FROM p79),
,p81 AS (SELECT * FROM p80),
,p82 AS (SELECT * FROM p81),
,p83 AS (SELECT * FROM p82),
,p84 AS (SELECT * FROM p83),
,p85 AS (SELECT * FROM p84),
,p86 AS (SELECT * FROM p85),
,p87 AS (SELECT * FROM p86),
,p88 AS (SELECT * FROM p87),
,p89 AS (SELECT * FROM p88),
,p90 AS (SELECT * FROM p89),
,p91 AS (SELECT * FROM p90),
,p92 AS (SELECT * FROM p91),
,p93 AS (SELECT * FROM p92),
,p94 AS (SELECT * FROM p93),
,p95 AS (SELECT * FROM p94),
,p96 AS (SELECT * FROM p95),
,p97 AS (SELECT * FROM p96),
,p98 AS (SELECT * FROM p97),
,p99 AS (SELECT * FROM p98)
SELECT
  COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p99 ON
  p99.ProductID = th.ProductID
WHERE
  p99.Name = 'Cable Lock'
```



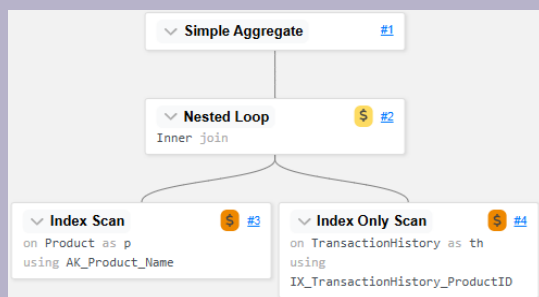
1000 CTEs!

```
...
,p993 AS (SELECT * FROM p992),
,p994 AS (SELECT * FROM p993),
,p995 AS (SELECT * FROM p994),
,p996 AS (SELECT * FROM p995),
,p997 AS (SELECT * FROM p996),
,p998 AS (SELECT * FROM p997),
,p999 AS (SELECT * FROM p998)
SELECT
  COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p999 ON
  p999.ProductID = th.ProductID
WHERE
  p999.Name = 'Cable Lock'
```

Can Postgres Handle It???

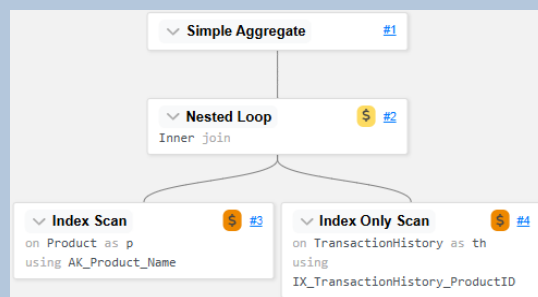
1 CTE

```
WITH p AS
(
  SELECT
    *
  FROM Product
)
SELECT
  COUNT(*)
FROM TransactionHistory AS th
INNER JOIN p ON
  p.ProductID = th.ProductID
WHERE
  p.Name = 'Cable Lock'
```



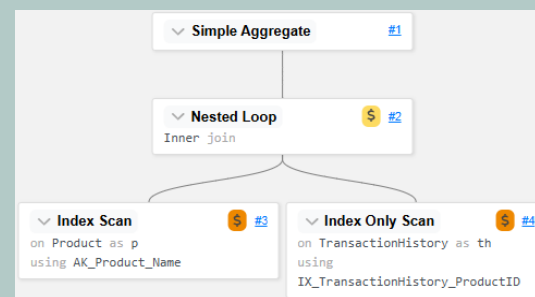
10 CTEs

```
WITH p0 AS
(
  SELECT
    *
  FROM Production.Product
)
,p1 AS (SELECT * FROM p0)
,p2 AS (SELECT * FROM p1)
,p3 AS (SELECT * FROM p2)
,p4 AS (SELECT * FROM p3)
,p5 AS (SELECT * FROM p4)
,p6 AS (SELECT * FROM p5)
,p7 AS (SELECT * FROM p6)
,p8 AS (SELECT * FROM p7)
,p9 AS (SELECT * FROM p8)
SELECT
  COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p9 ON
  p9.ProductID = th.ProductID
WHERE
  p9.Name = 'Cable Lock'
```



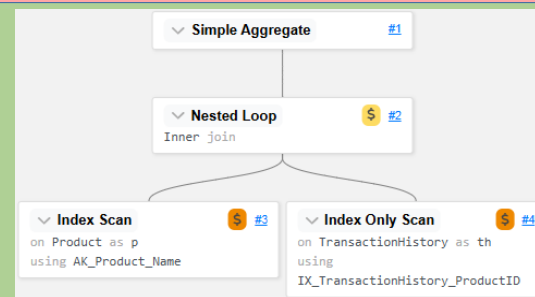
100 CTEs

```
...
,p72 AS (SELECT * FROM p71)
,p73 AS (SELECT * FROM p72)
,p74 AS (SELECT * FROM p73)
,p75 AS (SELECT * FROM p74)
,p76 AS (SELECT * FROM p75)
,p77 AS (SELECT * FROM p76)
,p78 AS (SELECT * FROM p77)
,p79 AS (SELECT * FROM p78)
,p80 AS (SELECT * FROM p79)
,p81 AS (SELECT * FROM p80)
,p82 AS (SELECT * FROM p81)
,p83 AS (SELECT * FROM p82)
,p84 AS (SELECT * FROM p83)
,p85 AS (SELECT * FROM p84)
,p86 AS (SELECT * FROM p85)
,p87 AS (SELECT * FROM p86)
,p88 AS (SELECT * FROM p87)
,p89 AS (SELECT * FROM p88)
,p90 AS (SELECT * FROM p89)
,p91 AS (SELECT * FROM p90)
,p92 AS (SELECT * FROM p91)
,p93 AS (SELECT * FROM p92)
,p94 AS (SELECT * FROM p93)
,p95 AS (SELECT * FROM p94)
,p96 AS (SELECT * FROM p95)
,p97 AS (SELECT * FROM p96)
,p98 AS (SELECT * FROM p97)
,p99 AS (SELECT * FROM p98)
SELECT
  COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p99 ON
  p99.ProductID = th.ProductID
WHERE
  p99.Name = 'Cable Lock'
```



1000 CTEs!

```
...
,p993 AS (SELECT * FROM p992)
,p994 AS (SELECT * FROM p993)
,p995 AS (SELECT * FROM p994)
,p996 AS (SELECT * FROM p995)
,p997 AS (SELECT * FROM p996)
,p998 AS (SELECT * FROM p997)
,p999 AS (SELECT * FROM p998)
,p1000 AS (SELECT * FROM p999)
,p1001 AS (SELECT * FROM p1000)
,p1002 AS (SELECT * FROM p1001)
,p1003 AS (SELECT * FROM p1002)
,p1004 AS (SELECT * FROM p1003)
,p1005 AS (SELECT * FROM p1004)
,p1006 AS (SELECT * FROM p1005)
,p1007 AS (SELECT * FROM p1006)
,p1008 AS (SELECT * FROM p1007)
,p1009 AS (SELECT * FROM p1008)
,p1010 AS (SELECT * FROM p1009)
,p1011 AS (SELECT * FROM p1010)
,p1012 AS (SELECT * FROM p1011)
,p1013 AS (SELECT * FROM p1012)
,p1014 AS (SELECT * FROM p1013)
,p1015 AS (SELECT * FROM p1014)
,p1016 AS (SELECT * FROM p1015)
,p1017 AS (SELECT * FROM p1016)
,p1018 AS (SELECT * FROM p1017)
,p1019 AS (SELECT * FROM p1018)
,p1020 AS (SELECT * FROM p1019)
,p1021 AS (SELECT * FROM p1020)
,p1022 AS (SELECT * FROM p1021)
,p1023 AS (SELECT * FROM p1022)
,p1024 AS (SELECT * FROM p1023)
,p1025 AS (SELECT * FROM p1024)
,p1026 AS (SELECT * FROM p1025)
,p1027 AS (SELECT * FROM p1026)
,p1028 AS (SELECT * FROM p1027)
,p1029 AS (SELECT * FROM p1028)
,p1030 AS (SELECT * FROM p1029)
,p1031 AS (SELECT * FROM p1030)
,p1032 AS (SELECT * FROM p1031)
,p1033 AS (SELECT * FROM p1032)
,p1034 AS (SELECT * FROM p1033)
,p1035 AS (SELECT * FROM p1034)
,p1036 AS (SELECT * FROM p1035)
,p1037 AS (SELECT * FROM p1036)
,p1038 AS (SELECT * FROM p1037)
,p1039 AS (SELECT * FROM p1038)
,p1040 AS (SELECT * FROM p1039)
,p1041 AS (SELECT * FROM p1040)
,p1042 AS (SELECT * FROM p1041)
,p1043 AS (SELECT * FROM p1042)
,p1044 AS (SELECT * FROM p1043)
,p1045 AS (SELECT * FROM p1044)
,p1046 AS (SELECT * FROM p1045)
,p1047 AS (SELECT * FROM p1046)
,p1048 AS (SELECT * FROM p1047)
,p1049 AS (SELECT * FROM p1048)
,p1050 AS (SELECT * FROM p1049)
,p1051 AS (SELECT * FROM p1050)
,p1052 AS (SELECT * FROM p1051)
,p1053 AS (SELECT * FROM p1052)
,p1054 AS (SELECT * FROM p1053)
,p1055 AS (SELECT * FROM p1054)
,p1056 AS (SELECT * FROM p1055)
,p1057 AS (SELECT * FROM p1056)
,p1058 AS (SELECT * FROM p1057)
,p1059 AS (SELECT * FROM p1058)
,p1060 AS (SELECT * FROM p1059)
,p1061 AS (SELECT * FROM p1060)
,p1062 AS (SELECT * FROM p1061)
,p1063 AS (SELECT * FROM p1062)
,p1064 AS (SELECT * FROM p1063)
,p1065 AS (SELECT * FROM p1064)
,p1066 AS (SELECT * FROM p1065)
,p1067 AS (SELECT * FROM p1066)
,p1068 AS (SELECT * FROM p1067)
,p1069 AS (SELECT * FROM p1068)
,p1070 AS (SELECT * FROM p1069)
,p1071 AS (SELECT * FROM p1070)
,p1072 AS (SELECT * FROM p1071)
,p1073 AS (SELECT * FROM p1072)
,p1074 AS (SELECT * FROM p1073)
,p1075 AS (SELECT * FROM p1074)
,p1076 AS (SELECT * FROM p1075)
,p1077 AS (SELECT * FROM p1076)
,p1078 AS (SELECT * FROM p1077)
,p1079 AS (SELECT * FROM p1078)
,p1080 AS (SELECT * FROM p1079)
,p1081 AS (SELECT * FROM p1080)
,p1082 AS (SELECT * FROM p1081)
,p1083 AS (SELECT * FROM p1082)
,p1084 AS (SELECT * FROM p1083)
,p1085 AS (SELECT * FROM p1084)
,p1086 AS (SELECT * FROM p1085)
,p1087 AS (SELECT * FROM p1086)
,p1088 AS (SELECT * FROM p1087)
,p1089 AS (SELECT * FROM p1088)
,p1090 AS (SELECT * FROM p1089)
,p1091 AS (SELECT * FROM p1090)
,p1092 AS (SELECT * FROM p1091)
,p1093 AS (SELECT * FROM p1092)
,p1094 AS (SELECT * FROM p1093)
,p1095 AS (SELECT * FROM p1094)
,p1096 AS (SELECT * FROM p1095)
,p1097 AS (SELECT * FROM p1096)
,p1098 AS (SELECT * FROM p1097)
,p1099 AS (SELECT * FROM p1098)
SELECT
  COUNT(*)
FROM Production.TransactionHistory AS th
INNER JOIN p999 ON
  p999.ProductID = th.ProductID
WHERE
  p999.Name = 'Cable Lock'
```



3 MINUTES LATER...

The Truth About CTEs

The Truth About CTEs

**Use CTEs
instead of
subqueries**

~~“For better
performance.”~~
...if you like to
read CTEs

The Truth About CTEs

**Use CTEs
instead of
subqueries**

~~“For better
performance.”~~
...if you like to
read CTEs

**Don't use too
many CTEs**

~~“The optimizer will
get confused.”~~
...you might get
confused!

Is the “Confusion” Based in Truth?

Is the “Confusion” Based in Truth?

```
WITH ranges AS
(
  SELECT
    entity_id,
    start_date,
    LEAD(start_date, 1, '9999-12-31')
      OVER (
        PARTITION BY entity_id
        ORDER BY start_date
      ) AS end_date
  FROM entity_dates
)
SELECT
  *
FROM ranges
WHERE
  '2025-06-03' >= start_date
  AND '2025-06-03' < end_date
```

Optimization Barriers

Optimization Barriers

Intraquery

Window Functions

Aggregation
(DISTINCT/GROUP BY)

TOP/LIMIT

Function calls on
indexed columns

Optimization Barriers

Intraquery

Window Functions

Aggregation
(DISTINCT/GROUP BY)

TOP/LIMIT

Function calls on
indexed columns

External

Table-Valued Functions
(Multistatement)

Temporary Tables

Postgres: CTEs set to
MATERIALIZE

No Optimizer Can Do This (Yet?)

```
WITH start_ends AS
(
  SELECT e_s.entity_id, MAX(e_s.start_date) AS the_date
  FROM entity_dates AS e_s
  WHERE
    e_s.start_date <= '2025-06-03'
  GROUP BY e_s.entity_id

  UNION ALL

  SELECT e_e.entity_id, MIN(e_e.start_date) AS the_date
  FROM entity_dates AS e_e
  WHERE
    e_e.start_date > '2025-06-03'
  GROUP BY e_e.entity_id

),
```

No Optimizer Can Do This (Yet?)

```
WITH start_ends AS
(
  SELECT e_s.entity_id, MAX(e_s.start_date) AS end_date
  FROM entity_dates AS e_s
  WHERE
    e_s.start_date <= '2025-06-03'
  GROUP BY e_s.entity_id

  UNION ALL

  SELECT e_e.entity_id, MIN(e_e.start_date) AS start_date
  FROM entity_dates AS e_e
  WHERE
    e_e.start_date > '2025-06-03'
  GROUP BY e_e.entity_id
),
```

```
ranges AS
(
  SELECT
    entity_id,
    the_date AS start_date,
    LEAD(the_date, 1, '9999-12-31')
      OVER (
        PARTITION BY entity_id
        ORDER BY the_date
      ) AS end_date
  FROM start_ends
)
SELECT
  *
FROM ranges
WHERE
  '2025-06-03' >= start_date
  AND '2025-06-03' < end_date
```

in | **wisdom** : Join Order

in wisdom : Join Order

SQL join order matters

“Write your queries from smallest table to largest table so that you can get the best performance”

Join Order: Let's Test It

```
SELECT
    th.TransactionID, p.Name,
    plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN ProductListPriceHistory AS plp ON
    plp.ProductID = p.ProductID
INNER JOIN TransactionHistory AS th ON
    th.ProductID = plp.ProductID
    AND th.ActualCost < plp.ListPrice
    AND th.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

Does it Matter, SQL Server?

Does it Matter, SQL Server?

```
SELECT
    th.TransactionID, p.Name,
    plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN ProductListPriceHistory AS plp ON
    plp.ProductID = p.ProductID
INNER JOIN TransactionHistory AS th ON
    th.ProductID = plp.ProductID
    AND th.ActualCost < plp.ListPrice
    AND th.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

```
SELECT
    th.TransactionID, p.Name,
    plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN TransactionHistory AS th ON
    th.ProductID = p.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
    plp.ProductID = th.ProductID
    AND th.ActualCost < plp.ListPrice
    AND th.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

```
SELECT
    th.TransactionID, p.Name,
    plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN Product AS p ON
    p.ProductID = plp.ProductID
INNER JOIN TransactionHistory AS th ON
    th.ProductID = p.ProductID
    AND th.ActualCost < plp.ListPrice
    AND th.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

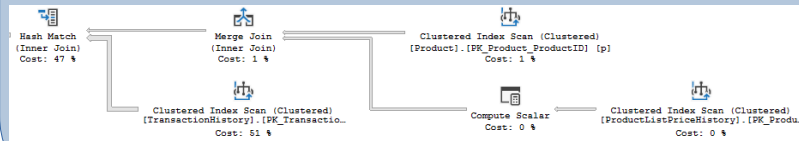
```
SELECT
    th.TransactionID, p.Name,
    plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN TransactionHistory AS th ON
    th.ProductID = plp.ProductID
    AND th.ActualCost < plp.ListPrice
    AND th.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
    p.ProductID = th.ProductID
```

```
SELECT
    th.TransactionID, p.Name,
    plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN Product AS p ON
    p.ProductID = th.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
    plp.ProductID = p.ProductID
    AND th.ActualCost < plp.ListPrice
    AND th.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

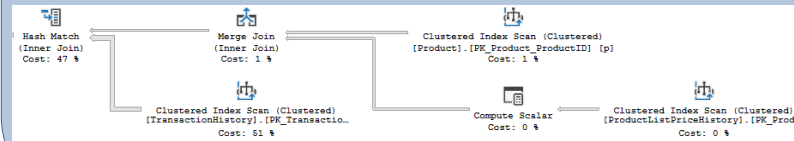
```
SELECT
    th.TransactionID, p.Name,
    plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN ProductListPriceHistory AS plp ON
    plp.ProductID = th.ProductID
    AND th.ActualCost < plp.ListPrice
    AND th.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
    p.ProductID = plp.ProductID
```

Does it Matter, SQL Server?

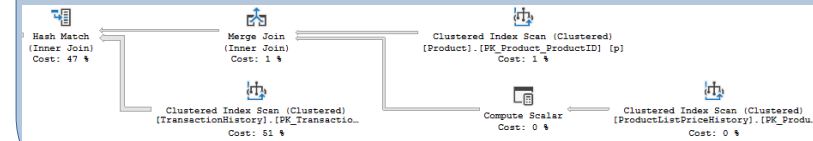
```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```



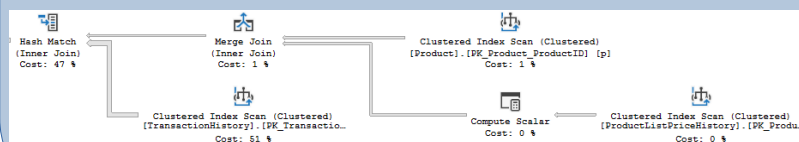
```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN TransactionHistory AS th ON
  th.ProductID = p.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = th.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```



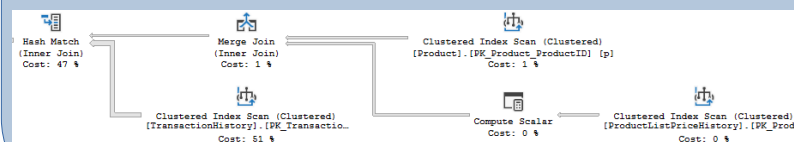
```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN Product AS p ON
  p.ProductID = plp.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = p.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```



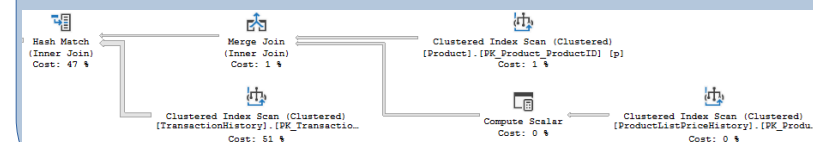
```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
```



```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```



```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = th.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
  p.ProductID = plp.ProductID
```



And Postgres..?

And Postgres..?

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN TransactionHistory AS th ON
  th.ProductID = p.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = th.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN Product AS p ON
  p.ProductID = plp.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = p.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

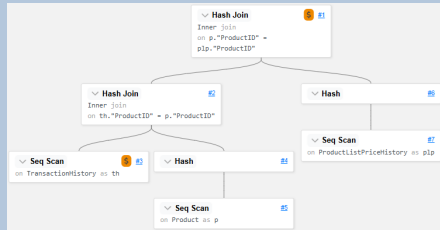
```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
```

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = th.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
  p.ProductID = plp.ProductID
```

And Postgres..?

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```



```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN TransactionHistory AS th ON
  th.ProductID = p.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = th.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN Product AS p ON
  p.ProductID = plp.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = p.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

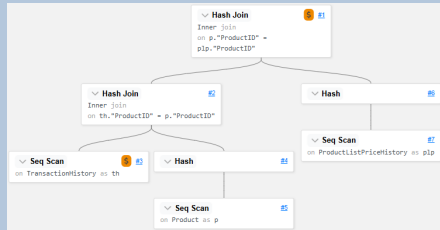
```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
```

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

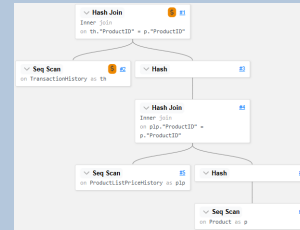
```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = th.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
  p.ProductID = plp.ProductID
```


And Postgres..?

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```



```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN TransactionHistory AS th ON
  th.ProductID = p.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = th.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```



```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN Product AS p ON
  p.ProductID = plp.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = p.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
```

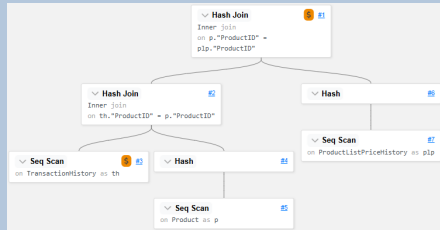
```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = th.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
  p.ProductID = plp.ProductID
```

And Postgres..?

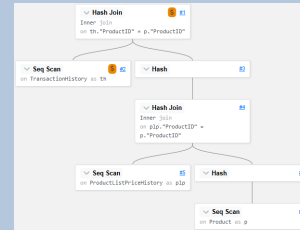
```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

1



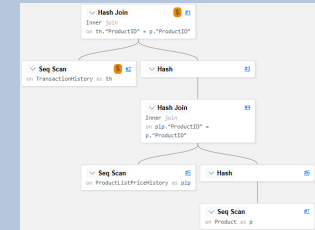
```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN TransactionHistory AS th ON
  th.ProductID = p.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = th.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

2



```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN Product AS p ON
  p.ProductID = plp.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = p.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

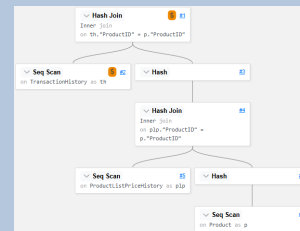
2



```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
```

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN Product AS p ON
  p.ProductID = th.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

2

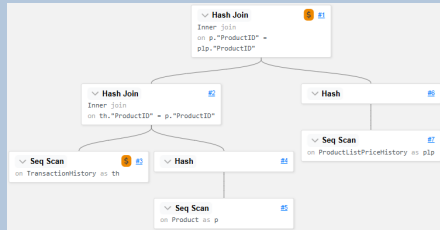


```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = th.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
  p.ProductID = plp.ProductID
```

And Postgres..?

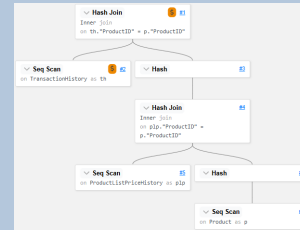
SELECT
th.TransactionID, p.Name,
plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN ProductListPriceHistory AS plp ON
plp.ProductID = p.ProductID
INNER JOIN TransactionHistory AS th ON
th.ProductID = plp.ProductID
AND th.ActualCost < plp.ListPrice
AND th.TransactionDate BETWEEN
plp.StartDate AND COALESCE(plp.EndDate, '99991231')

1



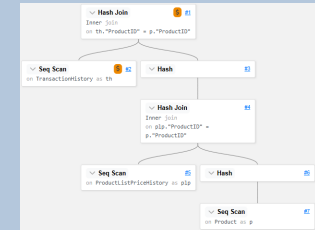
SELECT
th.TransactionID, p.Name,
plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN TransactionHistory AS th ON
th.ProductID = p.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
plp.ProductID = th.ProductID
AND th.ActualCost < plp.ListPrice
AND th.TransactionDate BETWEEN
plp.StartDate AND COALESCE(plp.EndDate, '99991231')

2



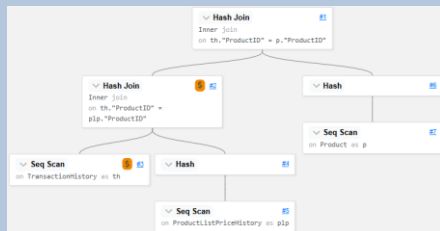
SELECT
th.TransactionID, p.Name,
plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN Product AS p ON
p.ProductID = plp.ProductID
INNER JOIN TransactionHistory AS th ON
th.ProductID = p.ProductID
AND th.ActualCost < plp.ListPrice
AND th.TransactionDate BETWEEN
plp.StartDate AND COALESCE(plp.EndDate, '99991231')

2



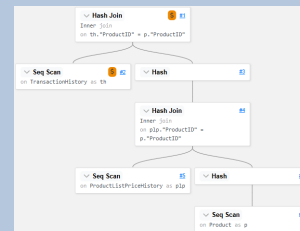
SELECT
th.TransactionID, p.Name,
plp.ListPrice, th.ActualCost
FROM ProductListPriceHistory AS plp
INNER JOIN TransactionHistory AS th ON
th.ProductID = plp.ProductID
AND th.ActualCost < plp.ListPrice
AND th.TransactionDate BETWEEN
plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
p.ProductID = th.ProductID

3



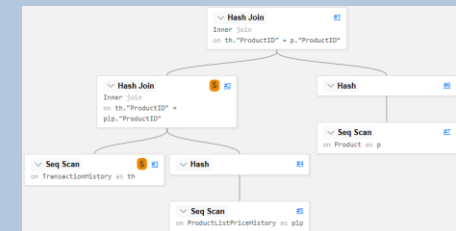
SELECT
th.TransactionID, p.Name,
plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN Product AS p ON
p.ProductID = th.ProductID
INNER JOIN ProductListPriceHistory AS plp ON
plp.ProductID = p.ProductID
AND th.ActualCost < plp.ListPrice
AND th.TransactionDate BETWEEN
plp.StartDate AND COALESCE(plp.EndDate, '99991231')

2



SELECT
th.TransactionID, p.Name,
plp.ListPrice, th.ActualCost
FROM TransactionHistory AS th
INNER JOIN ProductListPriceHistory AS plp ON
plp.ProductID = th.ProductID
AND th.ActualCost < plp.ListPrice
AND th.TransactionDate BETWEEN
plp.StartDate AND COALESCE(plp.EndDate, '99991231')
INNER JOIN Product AS p ON
p.ProductID = plp.ProductID

3



The Truth About Join Order

The Truth About Join Order

~~SQL~~Physical join order matters

~~“Write your queries from smallest table to largest table so that you can get the best performance”~~

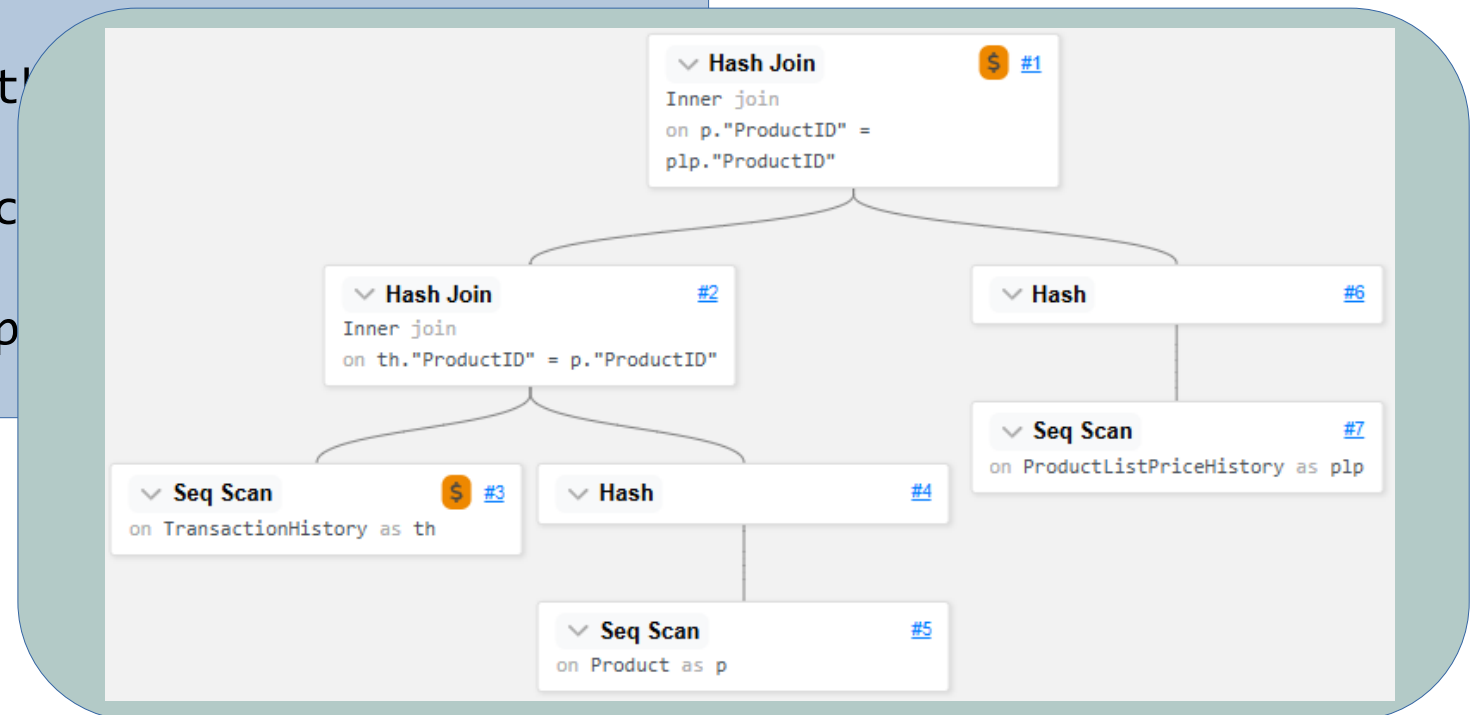
...to the optimizer. And it's totally controlled by the optimizer. Maybe..?

Join Order: A Closer Look (Shape 1)

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

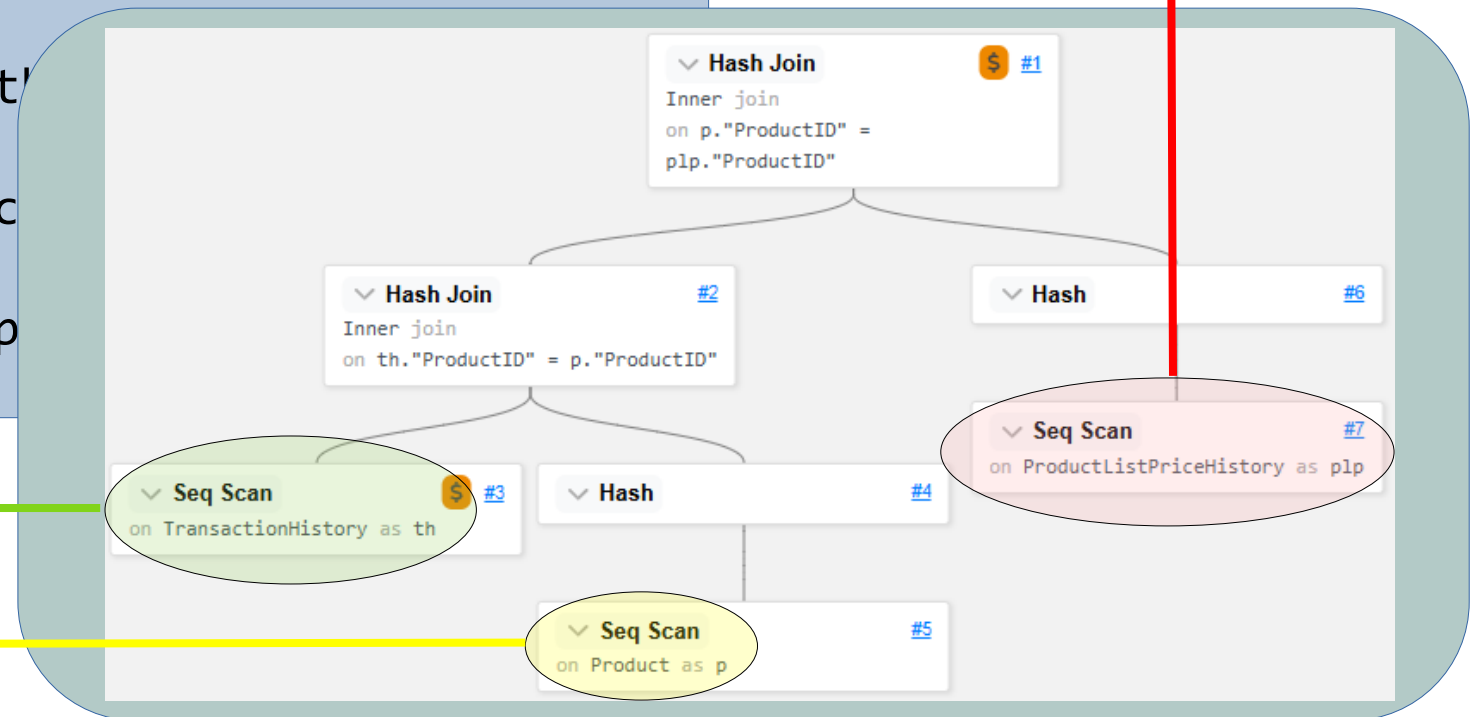
Join Order: A Closer Look (Shape 1)

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp
```



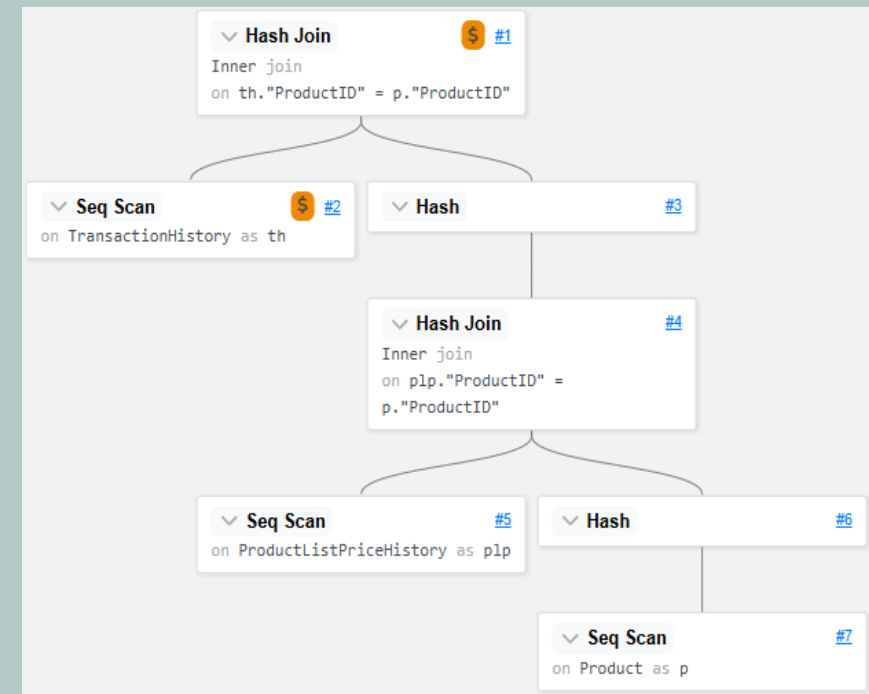
Join Order: A Closer Look (Shape 1)

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = p.ProductID
INNER JOIN TransactionHistory AS th ON
  th.ProductID = plp.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp
```



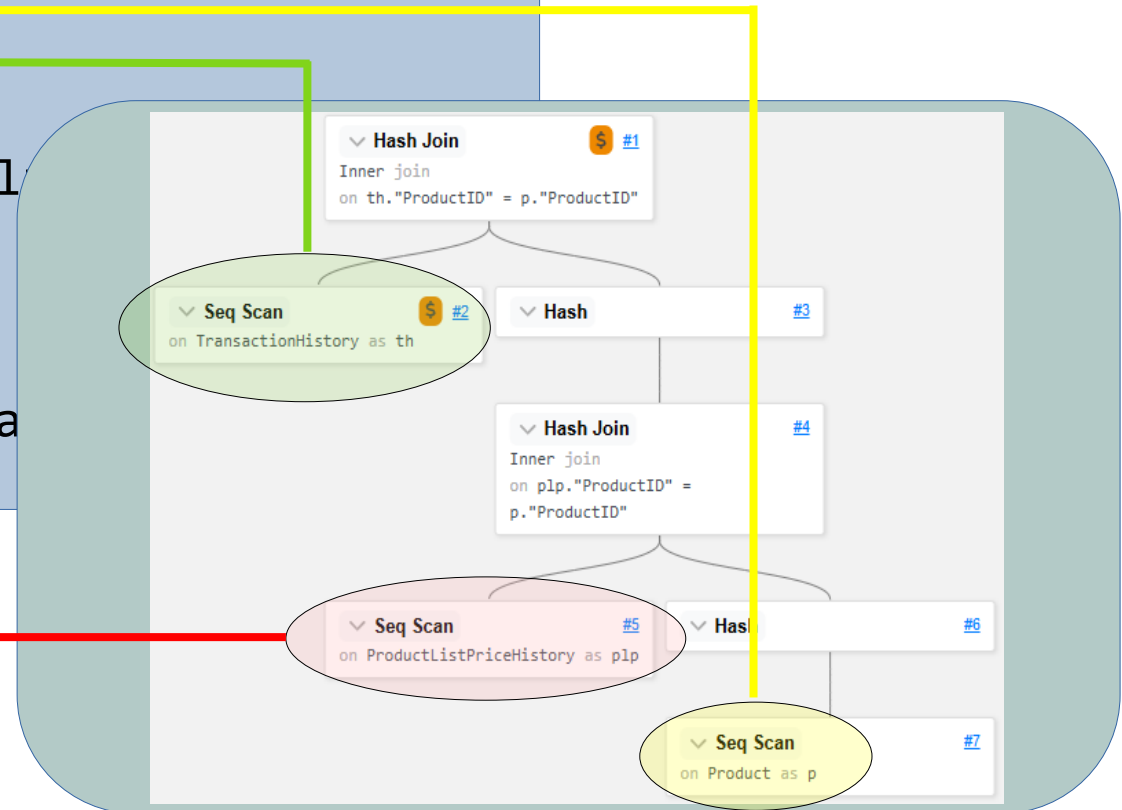
Join Order: A Closer Look (Shape 2)

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN TransactionHistory AS th ON
  th.ProductID = p.ProductID
INNER JOIN ProductListPriceHistory AS plp
  plp.ProductID = th.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate,
```



Join Order: A Closer Look (Shape 2)

```
SELECT
  th.TransactionID, p.Name,
  plp.ListPrice, th.ActualCost
FROM Product AS p
INNER JOIN TransactionHistory AS th ON
  th.ProductID = p.ProductID
INNER JOIN ProductListPriceHistory AS plp
  plp.ProductID = th.ProductID
  AND th.ActualCost < plp.ListPrice
  AND th.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate,
```



“I really want Product and TransactionHistory together
and also I want the query text to reflect the plan.”

“I really want Product and TransactionHistory together
and also I want the query text to reflect the plan.”

```
...  
OPTION (FORCE ORDER);
```

```
SET JOIN_COLLAPSE_LIMIT = 1;
```

```
...
```

“I really want Product and TransactionHistory together
and also I want the query text to reflect the plan.”

```
...  
OPTION (FORCE ORDER);
```

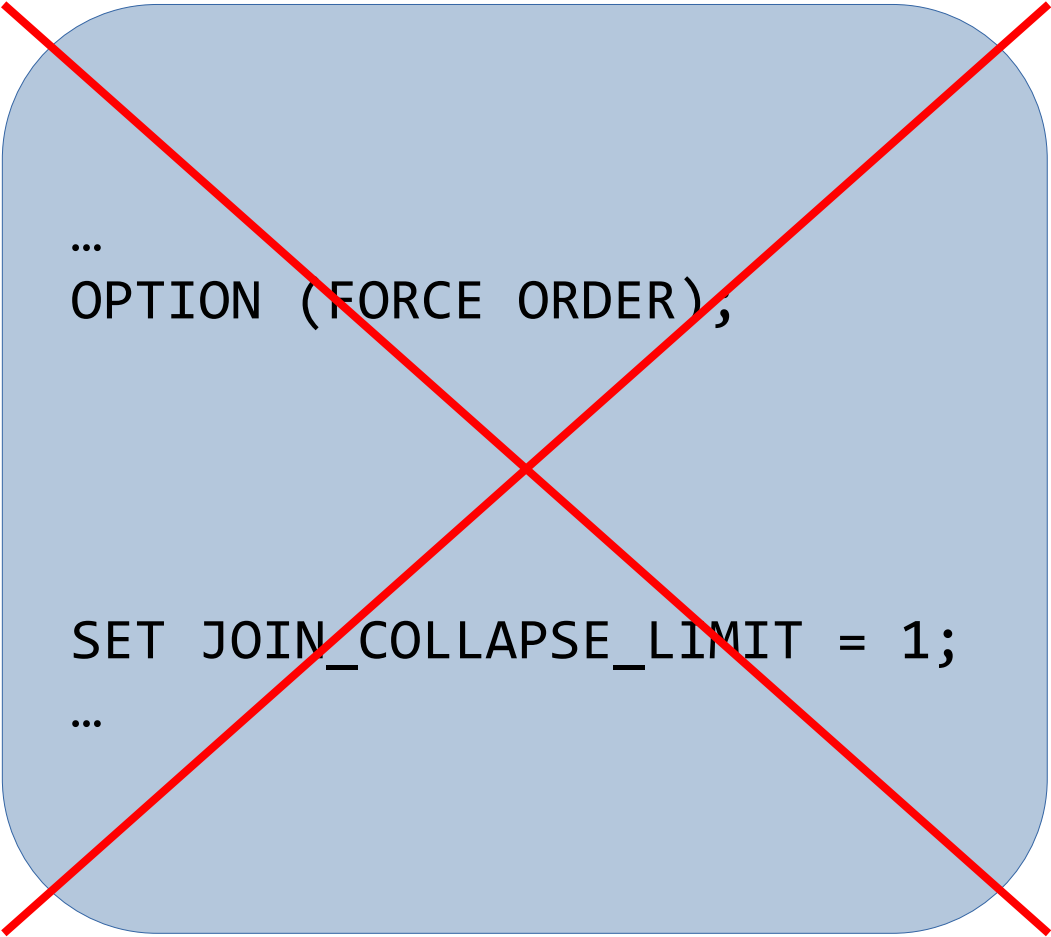
```
SET JOIN_COLLAPSE_LIMIT = 1;  
...
```

Large query?

View?

Table-valued function?

“I really want Product and TransactionHistory together
and also I want the query text to reflect the plan.”



```
...  
OPTION (FORCE ORDER),  
  
SET JOIN_COLLAPSE_LIMIT = 1;  
...
```

Large query?

View?

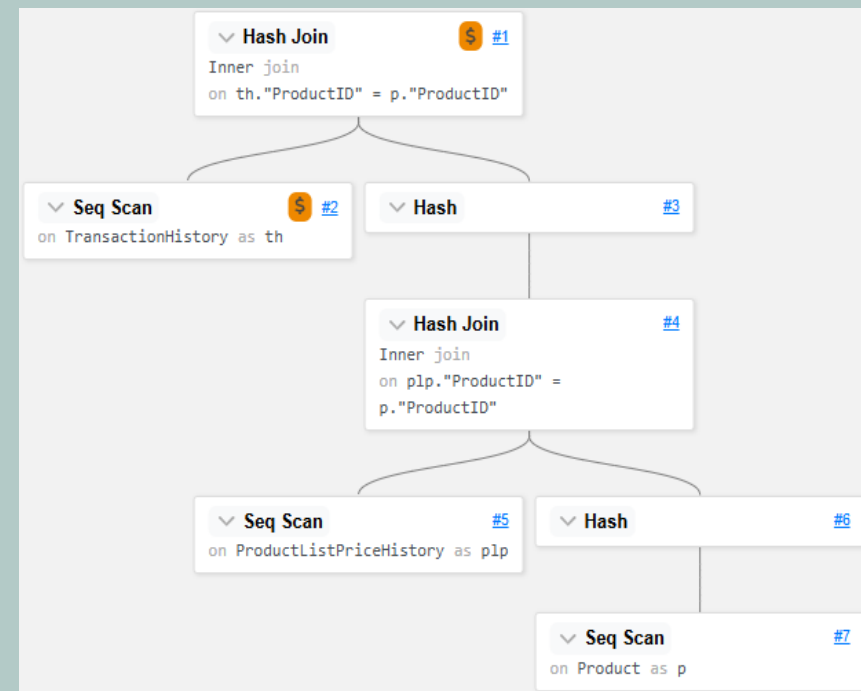
Table-valued function?

“I really want Product and TransactionHistory together and also I want the query text to reflect the plan.”

```
WITH pplp AS
(
    SELECT
        p.ProductID, p.Name,
        th.TransactionID, th.TransactionDate, th.ActualCost
    FROM Product AS p
    INNER JOIN TransactionHistory AS th ON
        th.ProductID = p.ProductID
)
SELECT
    pplp.TransactionID, pplp.Name,
    plp.ListPrice, pplp.ActualCost
FROM pplp
INNER JOIN ProductListPriceHistory AS plp ON
    plp.ProductID = pplp.ProductID
    AND pplp.ActualCost < plp.ListPrice
    AND pplp.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

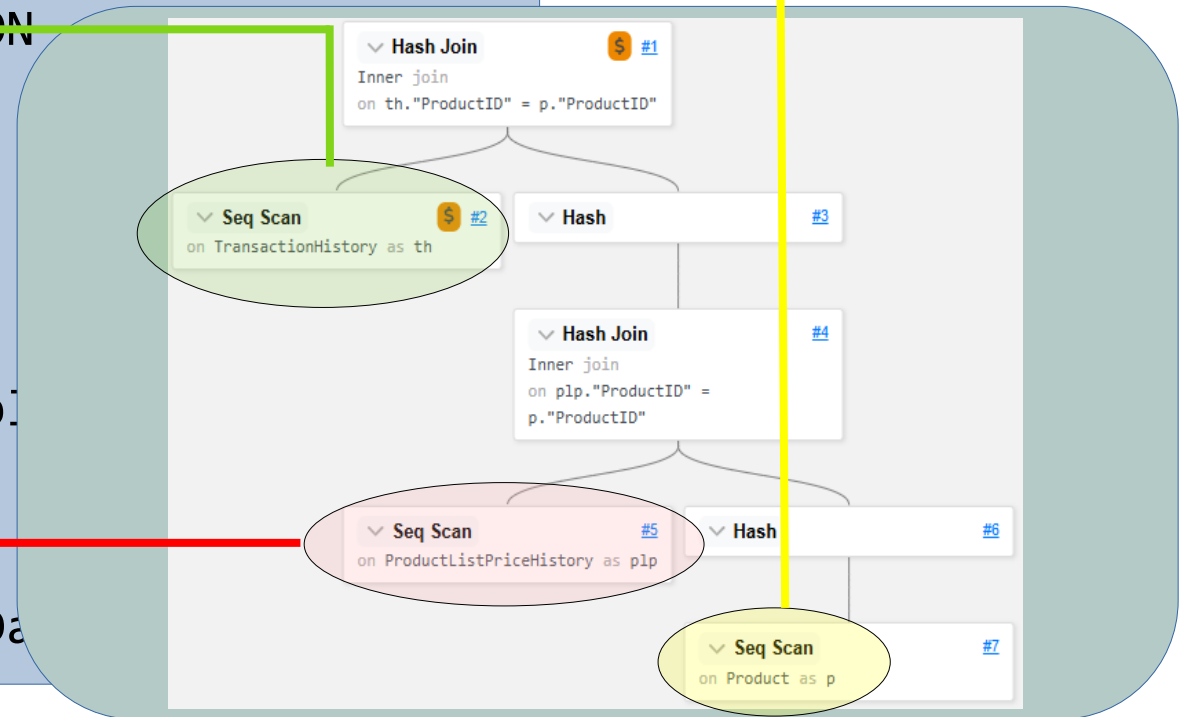
“I really want Product and TransactionHistory together and also I want the query text to reflect the plan.”

```
WITH pplp AS
(
    SELECT
        p.ProductID, p.Name,
        th.TransactionID, th.TransactionDate, th.ActualCost
    FROM Product AS p
    INNER JOIN TransactionHistory AS th ON
        th.ProductID = p.ProductID
)
SELECT
    pplp.TransactionID, pplp.Name,
    plp.ListPrice, pplp.ActualCost
FROM pplp
INNER JOIN ProductListPriceHistory AS plp
    plp.ProductID = pplp.ProductID
    AND pplp.ActualCost < plp.ListPrice
    AND pplp.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate,
```



“I really want Product and TransactionHistory together and also I want the query text to reflect the plan.”

```
WITH pplp AS
(
    SELECT
        p.ProductID, p.Name,
        th.TransactionID, th.TransactionDate, th.ActualCost
    FROM Product AS p
    INNER JOIN TransactionHistory AS th ON
        th.ProductID = p.ProductID
)
SELECT
    pplp.TransactionID, pplp.Name,
    plp.ListPrice, pplp.ActualCost
FROM pplp
INNER JOIN ProductListPriceHistory AS plp
    plp.ProductID = pplp.ProductID
    AND pplp.ActualCost < plp.ListPrice
    AND pplp.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, ...)
```

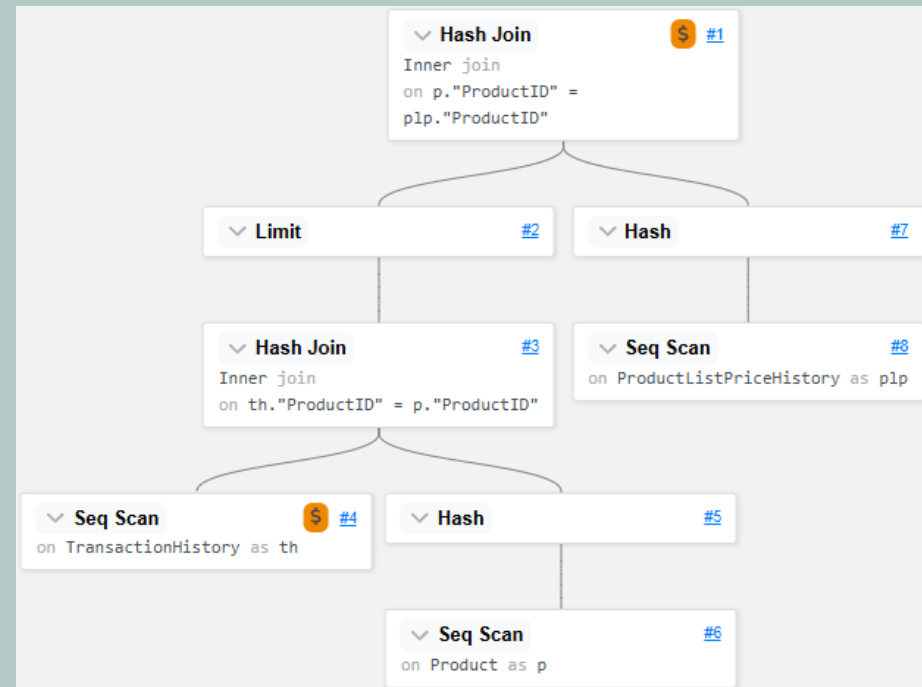


Purposeful Optimization Barrier!

```
WITH pplp AS
(
    SELECT
        p.ProductID, p.Name,
        th.TransactionID, th.TransactionDate, th.ActualCost
    FROM Product AS p
    INNER JOIN TransactionHistory AS th ON
        th.ProductID = p.ProductID
    LIMIT 2147483647
)
SELECT
    pplp.TransactionID, pplp.Name,
    plp.ListPrice, pplp.ActualCost
FROM pplp
INNER JOIN ProductListPriceHistory AS plp ON
    plp.ProductID = pplp.ProductID
    AND pplp.ActualCost < plp.ListPrice
    AND pplp.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

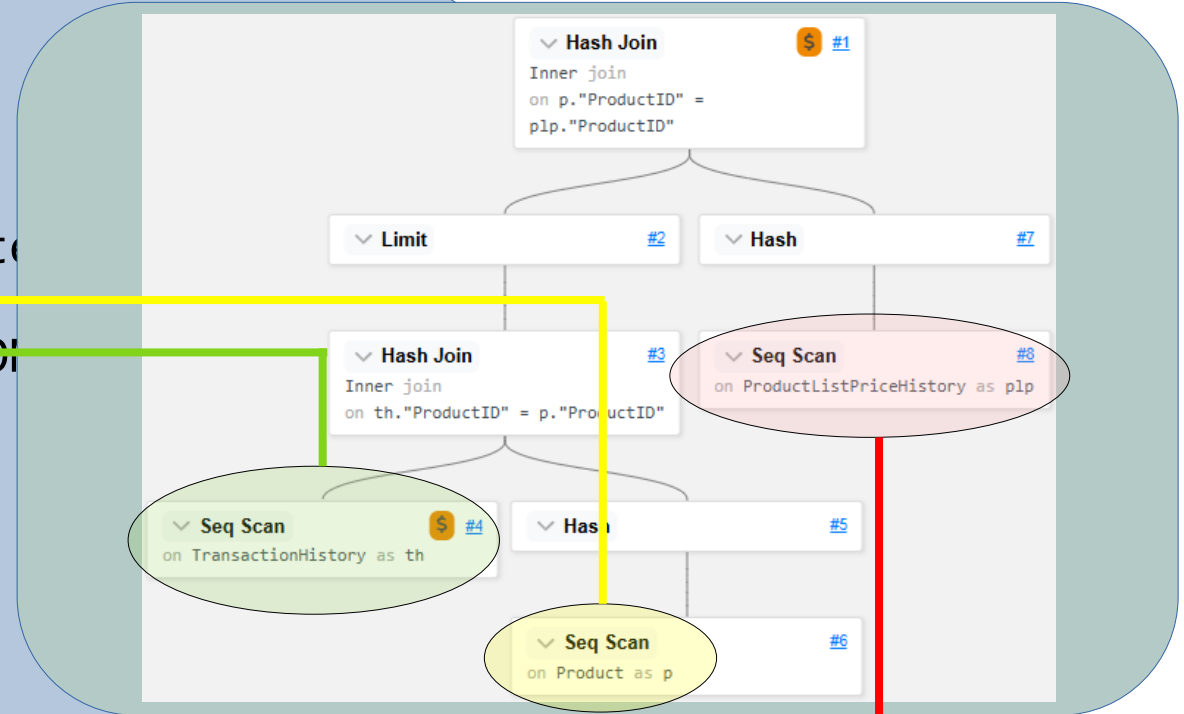
Purposeful Optimization Barrier!

```
WITH pplp AS
(
  SELECT
    p.ProductID, p.Name,
    th.TransactionID, th.TransactionDate
  FROM Product AS p
  INNER JOIN TransactionHistory AS th ON
    th.ProductID = p.ProductID
  LIMIT 2147483647
)
SELECT
  pplp.TransactionID, pplp.Name,
  plp.ListPrice, pplp.ActualCost
FROM pplp
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = pplp.ProductID
  AND pplp.ActualCost < plp.ListPrice
  AND pplp.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```



Purposeful Optimization Barrier!

```
WITH pplp AS
(
  SELECT
    p.ProductID, p.Name,
    th.TransactionID, th.TransactionDate
  FROM Product AS p
  INNER JOIN TransactionHistory AS th ON
    th.ProductID = p.ProductID
  LIMIT 2147483647
)
SELECT
  pplp.TransactionID, pplp.Name,
  plp.ListPrice, pplp.ActualCost
FROM pplp
INNER JOIN ProductListPriceHistory AS plp ON
  plp.ProductID = pplp.ProductID
  AND pplp.ActualCost < plp.ListPrice
  AND pplp.TransactionDate BETWEEN
    plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

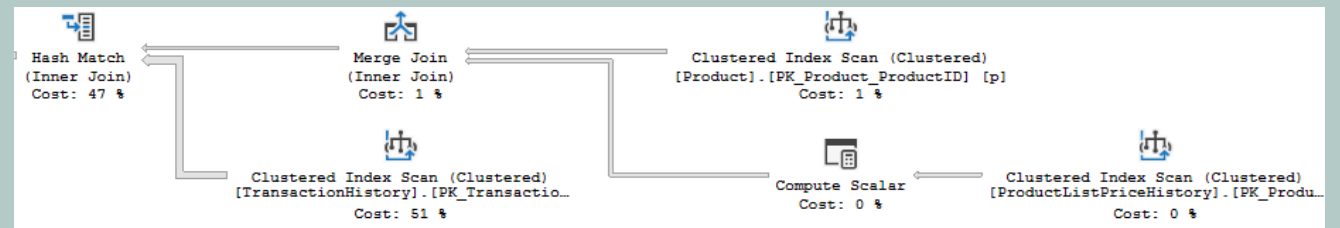


SQL Server: TOP

```
WITH pplp AS
(
    SELECT
        p.ProductID, p.Name,
        th.TransactionID, th.TransactionDate, th.ActualCost
    FROM Product AS p
    INNER JOIN TransactionHistory AS th ON
        th.ProductID = p.ProductID
)
SELECT
    pplp.TransactionID, pplp.Name,
    plp.ListPrice, pplp.ActualCost
FROM pplp
INNER JOIN ProductListPriceHistory AS plp ON
    plp.ProductID = pplp.ProductID
    AND pplp.ActualCost < plp.ListPrice
    AND pplp.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

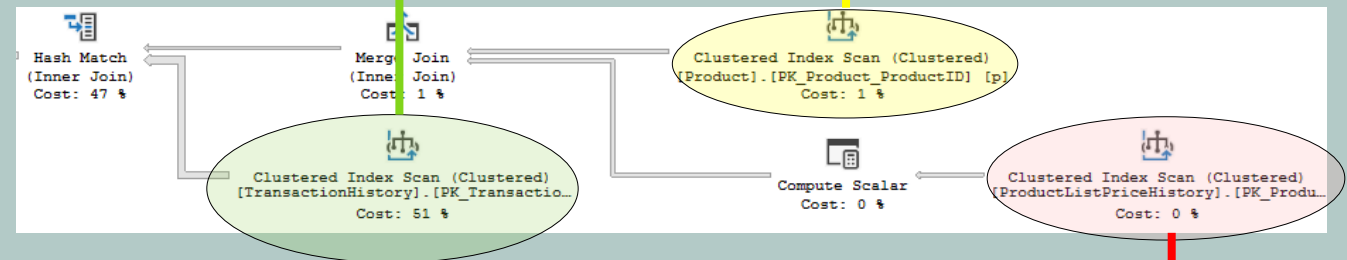
SQL Server: TOP

```
WITH pplp AS
(
    SELECT
        p.ProductID, p.Name,
        th.TransactionID, th.TransactionDate
    FROM Product AS p
    INNER JOIN TransactionHistory AS th
        th.ProductID = p.ProductID
)
SELECT
    pplp.TransactionID, pplp.Name,
    plp.ListPrice, pplp.ActualCost
FROM pplp
INNER JOIN ProductListPriceHistory AS plp
    pplp.ProductID = pplp.ProductID
    AND pplp.ActualCost < plp.ListPrice
    AND pplp.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```



SQL Server: TOP

```
WITH pplp AS
(
    SELECT
        p.ProductID, p.Name,
        th.TransactionID, th.TransactionDate
    FROM Product AS p
    INNER JOIN TransactionHistory AS th
        th.ProductID = p.ProductID
)
SELECT
    pplp.TransactionID, pplp.Name,
    plp.ListPrice, pplp.ActualCost
FROM pplp
INNER JOIN ProductListPriceHistory AS plp
    pplp.ProductID = pplp.ProductID
    AND pplp.ActualCost < plp.ListPrice
    AND pplp.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

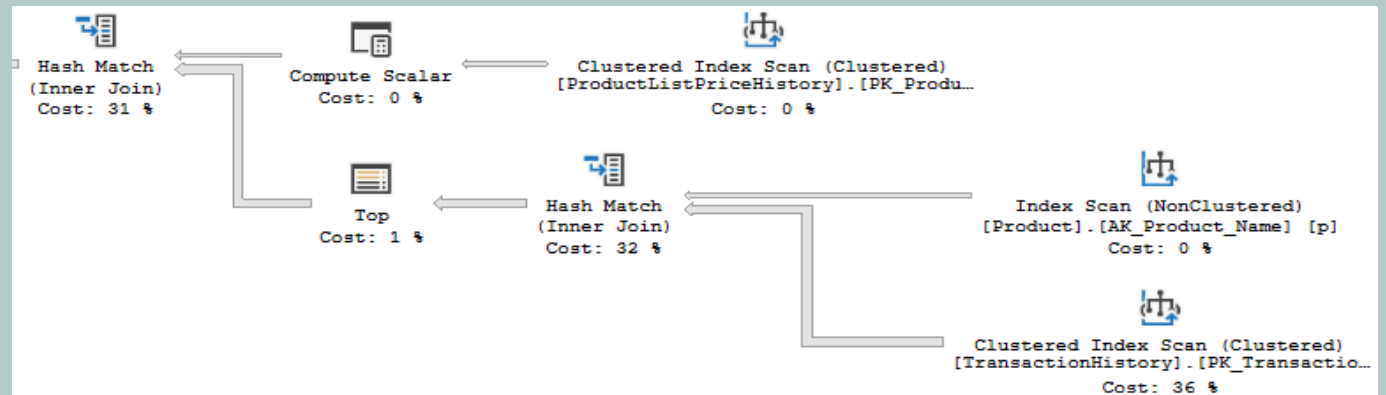


SQL Server: TOP

```
WITH pplp AS
(
    SELECT TOP(2147483647)
        p.ProductID, p.Name,
        th.TransactionID, th.TransactionDate, th.ActualCost
    FROM Product AS p
    INNER JOIN TransactionHistory AS th ON
        th.ProductID = p.ProductID
)
SELECT
    pplp.TransactionID, pplp.Name,
    plp.ListPrice, pplp.ActualCost
FROM pplp
INNER JOIN ProductListPriceHistory AS plp ON
    plp.ProductID = pplp.ProductID
    AND pplp.ActualCost < plp.ListPrice
    AND pplp.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, '99991231')
```

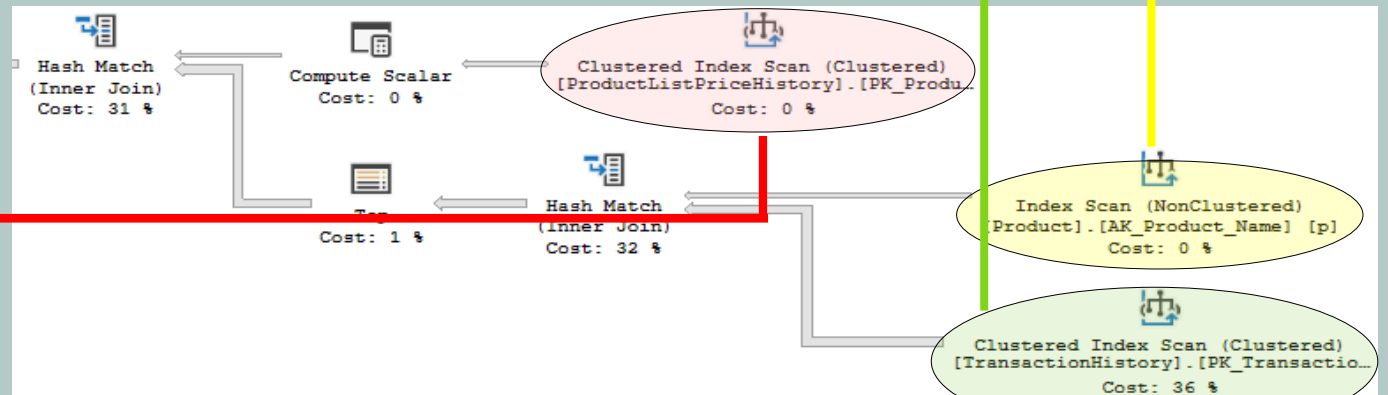

SQL Server: TOP

```
WITH pplp AS
(
    SELECT TOP(2147483647)
        p.ProductID, p.Name,
        th.TransactionID, th.TransactionDate, th.ActualCost
    FROM Product AS p
    INNER JOIN TransactionHistory AS th ON
        th.ProductID = p.ProductID
)
SELECT
    pplp.TransactionID, pplp.Name,
    plp.ListPrice, pplp.ActualCost
FROM pplp
INNER JOIN ProductListPriceHistory
    plp.ProductID = pplp.ProductID
    AND pplp.ActualCost < plp.ListPrice
    AND pplp.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, 99991231)
```



SQL Server: TOP

```
WITH pplp AS
(
    SELECT TOP(2147483647)
        p.ProductID, p.Name,
        th.TransactionID, th.TransactionDate, th.ActualCost
    FROM Product AS p
    INNER JOIN TransactionHistory AS th ON
        th.ProductID = p.ProductID
)
SELECT
    pplp.TransactionID, pplp.Name,
    plp.ListPrice, pplp.ActualCost
FROM pplp
INNER JOIN ProductListPriceHistory
    plp.ProductID = pplp.ProductID
    AND pplp.ActualCost < plp.ListPrice
    AND pplp.TransactionDate BETWEEN
        plp.StartDate AND COALESCE(plp.EndDate, 99991231)
```



Other Common Issues

Other Common Issues

Multiple similar
subqueries in the
SELECT list

Rewrite using
LATERAL or
APPLY
(or just a join!)

```
SELECT
  x.id,
  (
    SELECT
      tbl2.name
    FROM tbl2
    WHERE tbl2.col = x.col
  ) AS name,
  (
    SELECT
      tbl2.description
    FROM tbl2
    WHERE tbl2.col = x.col
  ) AS description
FROM tbl1 AS x
```

Other Common Issues

Multiple similar
subqueries in the
SELECT list

Rewrite using
LATERAL or
APPLY
(or just a join!)

```
SELECT
  x.id,
  (
    SELECT
      tbl2.name
    FROM tbl2
    WHERE tbl2.col = x.col
  ) AS name,
  (
    SELECT
      tbl2.description
    FROM tbl2
    WHERE tbl2.col = x.col
  ) AS description
FROM tbl1 AS x
```

```
SELECT
  x.id,
  y.name,
  y.description
FROM tbl1 AS x
OUTER JOIN LATERAL
(
  SELECT
    tbl2.name,
    tbl2.description
  FROM tbl2
  WHERE
    tbl2.col = x.col
) AS y ON TRUE
```

Other Common Issues

Massive
GROUP BY

Group
sub-sections
individually in a
CTE/derived table

```
SELECT
  y.name,
  y.description,
  y.price,
  SUM(z.total) AS total,
  MAX(z.date) AS last_date
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  tbl2.col = tbl1.col
INNER JOIN tbl3 AS z ON
  tbl3.colx = tbl2.colx
GROUP BY
  y.name,
  y.description,
  y.price
```

Other Common Issues

Massive
GROUP BY

Group
sub-sections
individually in a
CTE/derived table

```
SELECT
  y.name,
  y.description,
  y.price,
  SUM(z.total) AS total,
  MAX(z.date) AS last_date
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  tbl2.col = tbl1.col
INNER JOIN tbl3 AS z ON
  tbl3.colx = tbl2.colx
GROUP BY
  y.name,
  y.description,
  y.price
```

```
SELECT
  y.name,
  y.description,
  y.price,
  z.*
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  tbl2.col = tbl1.col
CROSS JOIN LATERAL
(
  SELECT
    SUM(tbl3.total) AS total,
    MAX(tbl3.date) AS last_date
  FROM tbl3
  WHERE
    tbl3.colx = y.colx
) AS z
```

Other Common Issues

DISTINCT
on top of a large
query

See GROUP BY...

```
SELECT DISTINCT
  x.col,
  y.colx,
  y.name,
  y.description,
  y.price,
  z.total,
  z.date
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  tbl2.col = tbl1.col
INNER JOIN tbl3 AS z ON
  tbl3.colx = tbl2.colx
```


Other Common Issues

DISTINCT
on top of a large
query

See GROUP BY...

```
SELECT DISTINCT
  x.col,
  y.colx,
  y.name,
  y.description,
  y.price,
  z.total,
  z.date
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  tbl2.col = tbl1.col
INNER JOIN tbl3 AS z ON
  tbl3.colx = tbl2.colx
```

```
SELECT
  x.col,
  y.colx,
  y.name,
  y.description,
  y.price,
  z.*
FROM tbl1 AS x
INNER JOIN tbl2 AS y ON
  tbl2.col = tbl1.col
CROSS JOIN LATERAL
(
  SELECT DISTINCT
    z.total, z.date
  FROM tbl3 AS z
  WHERE
    tbl3.colx = tbl2.colx
) AS z
```

Approaching Tuning Scientifically

Approaching Tuning Scientifically

Focus

Break your big query into small parts. Tune each separately.

Approaching Tuning Scientifically

Focus

Break your big query into small parts. Tune each separately.

Test and Iterate

Make a small change. Test.
Make another small change. Test again.

Approaching Tuning Scientifically

Focus

Break your big query into small parts. Tune each separately.

Test and Iterate

Make a small change. Test. Make another small change. Test again.

Take Notes

Keep track of what works and what doesn't. Try to understand why, for next time.

Remember: Test Carefully!

Remember: Test Carefully!

Data caching?
Statistics?

Was the last test
run on a cold
cache? Did
statistics update?

Remember: Test Carefully!

Data caching?
Statistics?

Was the last test
run on a cold
cache? Did
statistics update?

Server activity?

Was the server
super busy earlier?
Did you check for
blocking?

Remember: Test Carefully!

Data caching?
Statistics?

Was the last test
run on a cold
cache? Did
statistics update?

Server activity?

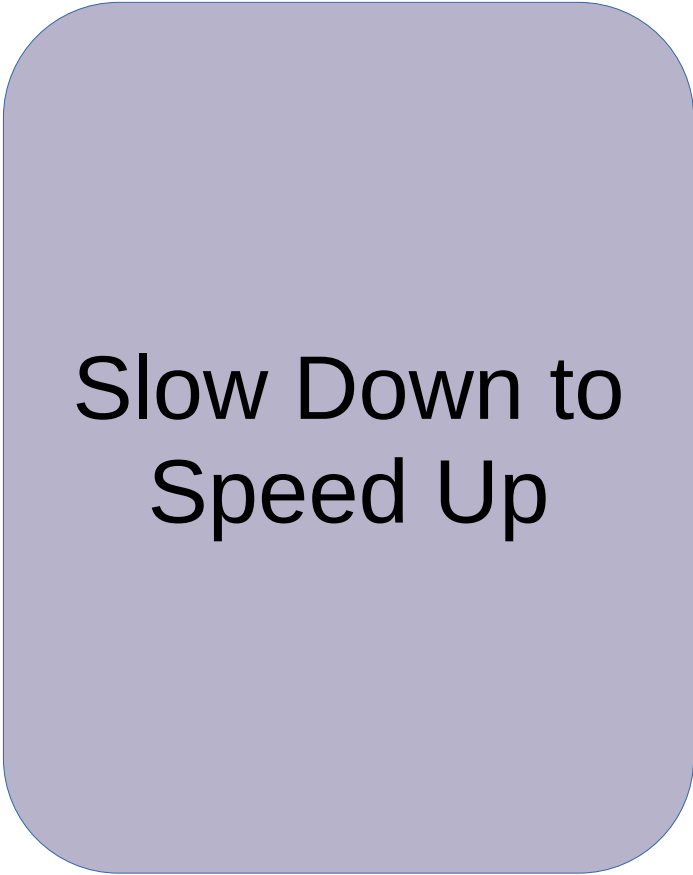
Was the server
super busy earlier?
Did you check for
blocking?

Changed output?

Maybe the query
really is faster...but
did you triple-check
the logic?

Summary

Summary



Slow Down to
Speed Up

Summary

Slow Down to
Speed Up

Study the
Edges of Your
Optimizer of
Choice

Summary

Slow Down to
Speed Up

Study the
Edges of Your
Optimizer of
Choice



Your feedback is important to us



Evaluate this session at:

passdatacommunitysummit.com/evaluations

Thank you

Adam Machanic



amachanic@gmail.com



[linkedin.com/in/adammachanic](https://www.linkedin.com/in/adammachanic)